

IT3101 – Data Warehousing and Business Intelligence



Group Assignment

Group ID: 2026-DS-27

Design and Implementation of a Data Warehouse and Business Intelligence Solution

Student ID	Name
IT24101566	Madhusanka K.B.D.
IT24101315	Egalla J.I.
IT24101675	Nanayakkara S.N.N.
IT24100706	Denipitiyage C.N.

October 2026

Contents

1	Introduction	4
2	Project Objectives	4
3	Task 1: Dataset Selection and Business Scenario Identification	4
3.1	Business Domain	4
3.2	Company and System Overview	4
3.3	Purpose of the Operational System	4
3.4	Existing Business Challenges	5
3.5	Proposed DW/BI Solution Justification	6
3.6	Dataset Source	6
3.7	Dataset Format and Structure	6
3.8	Dataset Relationships	6
3.9	Dataset Size	7
3.10	Attribute Description	7
3.11	Dataset Suitability for DW/BI Implementation	7
4	Task 2: Data Source Identification and Preparation	8
4.1	Data Source Inventory	8
4.2	Source System Description	8
4.2.1	Orders Source	8
4.2.2	Order Items Source	9
4.2.3	Customer Source	9
4.2.4	Product Source	9
4.2.5	Seller Source	9
4.2.6	Payment Source	9
4.2.7	Review Source	10
4.3	Source Integration Relationships	10
4.4	Data Quality Assessment	10
4.4.1	Missing Values	10
4.4.2	Duplicate Records	11
4.4.3	Inconsistent Categories	11
4.4.4	Data Type Issues	11
4.4.5	Incomplete Delivery Information	11
4.4.6	Geolocation Duplicate Records	11
4.4.7	Inconsistent Seller City Values	11
4.4.8	Outlier Presence in Numeric Measures	11
4.5	Data Cleaning and Transformation Requirements	12
4.6	Staging Area Design	12
4.7	Source-to-Staging Mapping	13
4.8	Data Preparation	14
4.8.1	Handling Complex Review Text Data	14
4.9	Staging Validation	17
4.10	Task 2 Outcome	19
5	Task 3: Data Warehouse Architecture Design	19
5.1	Architecture Overview	19
5.2	Data Source Layer	20
5.3	Staging Area and ETL (Data Integration Layer)	20
5.3.1	Staging Area (Raw Buffer)	20
5.3.2	ETL Processing Pipeline	21
5.3.3	Enforced Data Quality Rules	21
5.4	Data Storage Layer	21

5.4.1	Enterprise Data Warehouse (Star Schema)	21
5.4.2	Subject Area Data Marts	22
5.5	Analytics and BI (Presentation Layer)	22
5.5.1	Reporting Tools	22
5.5.2	OLAP Tools (Multidimensional)	22
5.5.3	Power BI Dashboards	22
5.6	Summary of Layer Interactions	23
6	Task 4: Dimensional Data Warehouse Design and Implementation	23
6.1	Business Process	23
6.2	Fact Table Grain	23
6.3	The Grain-Mismatch Problem and How It Is Resolved	23
6.4	Measures	24
6.5	Dimension Tables and Foreign Keys	24
6.6	Star Schema Diagram	25
6.7	Design Assumptions	26
6.8	Warehouse Implementation	26
7	Task 5: ETL Development	29
7.1	ETL Overview	29
7.2	Extraction Method and Source Connection	29
7.3	Staging Layer Design	30
7.4	Data Profiling and Cleaning Rules	30
7.5	ETL Workflow	31
7.6	Source-to-Target Mapping	33
7.7	Load Order and Re-runability	34
7.8	Dim_Date Generation	34
7.9	Dim_Location Transformation	34
7.10	Dimension Load Validation	34
7.11	Fact Table Load	36
7.12	Fact Load Results	37
7.13	Reconciliation with the Staging Layer	38
7.14	Integrity Validation	39
7.15	Summary	40
8	Task 6: Data Mart Development	40
8.1	Purpose of the Data Mart	40
8.2	Intended Users and Business Questions	41
8.3	Design	41
8.4	Implementation	42
8.5	Access Control	43
8.6	Validation	43
8.7	Example Analyses	45
8.8	Benefits	47
8.9	Limitations and Extensions	47
9	Task 7: OLAP Analysis and Business Intelligence Dashboard Development	47
9.1	Overview	47
9.2	Data Model	48
9.3	DAX Measures	48
9.4	Report Page 1: Executive Summary	48
9.5	Report Page 2: Trend Analysis	49
9.6	Report Page 3: Interactive Analysis	50
9.6.1	OLAP Operations Demonstrated	51
9.7	Limitations	53

10 Task 8: Business Insights and Recommendations	53
10.1 Insight 1: Strong year-over-year growth	53
10.2 Insight 2: A clear year-end seasonal peak	54
10.3 Insight 3: Revenue is concentrated in one region	54
10.4 Insight 4: Revenue is spread across many categories	54
10.5 Insight 5: Customers mostly buy a single item	54
10.6 Insight 6: Monthly growth levelled off in 2018	54
10.7 How the Insights Support Decision-Making	55
10.8 Limitations	55
11 Conclusion	55
11.1 Summary of Work	55
11.2 Key Findings	55
11.3 Limitations	55
11.4 Future Work	55
12 References	56

1 Introduction

This report presents the design and implementation of an end-to-end Data Warehouse and Business Intelligence solution. The project covers dataset selection, source data preparation, warehouse architecture design, dimensional modelling, ETL development, data mart implementation, BI dashboard development, and business insights generation.

2 Project Objectives

The objectives of this project are:

- To analyse an operational e-commerce dataset and identify analytical requirements.
- To design a dimensional data warehouse model.
- To develop ETL processes for data integration and transformation.
- To create analytical data marts supporting business decision-making.
- To develop interactive BI dashboards using Power BI.
- To generate actionable business insights from historical transaction data.
- To evaluate the effectiveness of the BI solution in supporting data-driven business decision-making.

3 Task 1: Dataset Selection and Business Scenario Identification

3.1 Business Domain

E-commerce / Online Retail Marketplace — specifically a multi-vendor online marketplace connecting independent sellers with customers across Brazil.

3.2 Company and System Overview

Olist acts as an intermediary platform connecting independent sellers with customers, providing marketplace services including order management, payment integration, and logistics coordination.

3.3 Purpose of the Operational System

The selected dataset represents an anonymised extract of an operational e-commerce marketplace system operated by Olist. Such a system is typically organised around a set of interconnected operational entities:

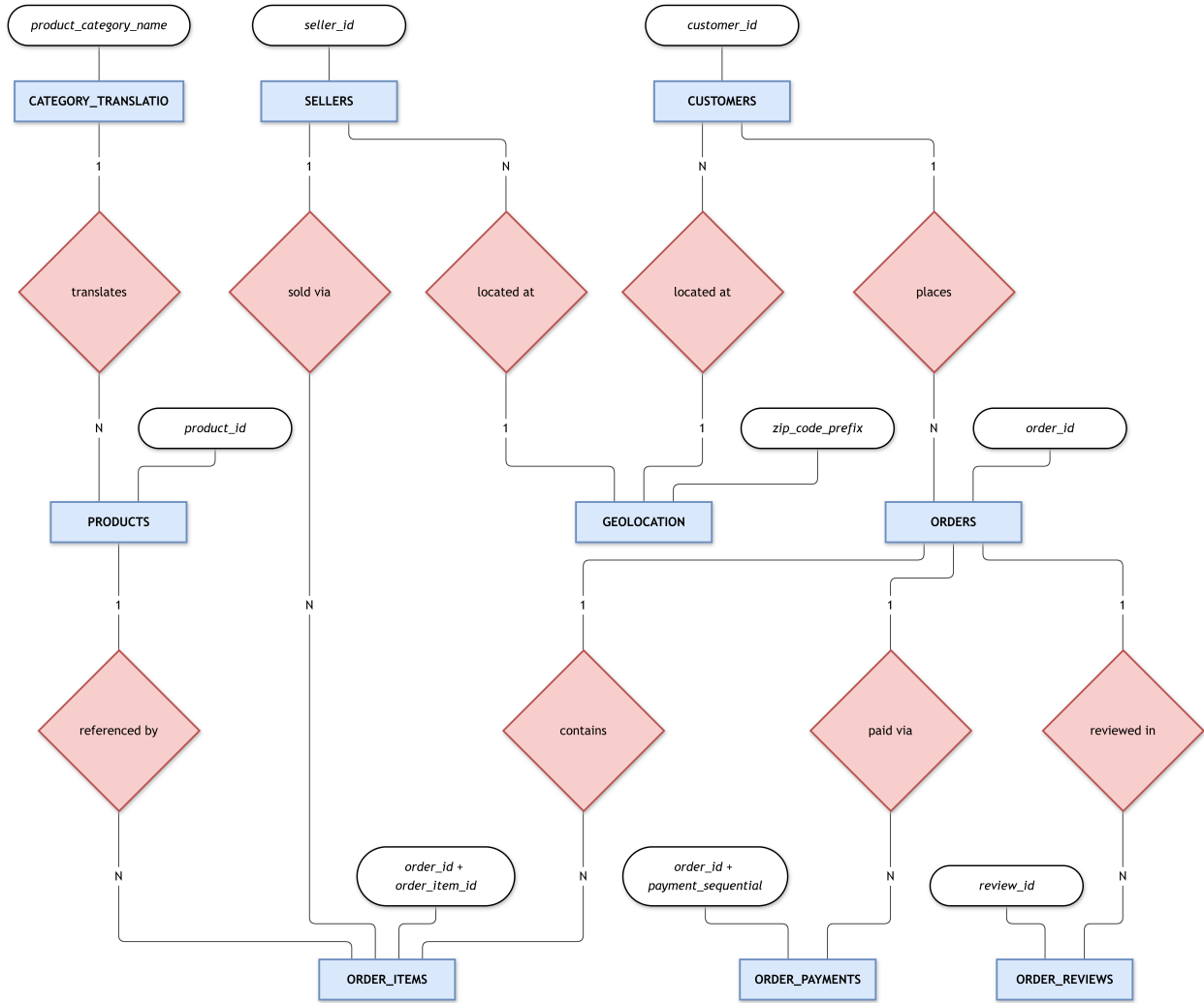


Figure 1: Entity-relationship diagram of the Olist source tables

Here, **Order Items** acts as the bridging entity between orders, products, and sellers, since each order can contain multiple products supplied by different sellers. **Payments**, **Reviews**, and **Delivery** are each independently associated with the parent **Orders** entity. These operational components are optimised for recording individual transactions accurately and quickly (transaction processing), rather than for answering broad analytical questions across large volumes of historical data. This distinction is the primary motivation for building a separate data warehouse and BI layer.

3.4 Existing Business Challenges

Because an operational system of this kind is optimised for transaction processing rather than analysis, answering management-level questions directly against it would require complex multi-table joins, repeated ad-hoc querying, and would risk degrading live platform performance. Typical questions that are difficult to answer efficiently include:

- Which product categories and sellers drive the most revenue, and how is this trending over time?
- Which Brazilian states/regions experience the longest delivery delays, and does this correlate with lower review scores?
- What payment types and instalment plans do customers prefer, and how do they relate to order value?

- Who are the most valuable customers, and how does satisfaction (review score) evolve month over month?
- Are there seasonal patterns in order volume that should inform inventory or marketing decisions?

Without an integrated analytical environment, management decisions rely on fragmented operational reports, making it difficult to identify sales opportunities, improve customer satisfaction, and optimise logistics performance.

3.5 Proposed DW/BI Solution Justification

A dimensional data warehouse integrates data from multiple operational sources into a structure optimised for efficient analytical querying and reporting — allowing the business questions above to be answered through interactive dashboards rather than one-off queries against operational tables.

3.6 Dataset Source

Brazilian E-Commerce Public Dataset by Olist, publicly available on Kaggle:

<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

This dataset contains anonymised commercial transaction data publicly released by Olist for research and educational purposes.

3.7 Dataset Format and Structure

The dataset consists of nine separate CSV files, representing a normalised relational structure similar to an OLTP database, linked through common key fields. CSV format was selected as the source format because it represents typical flat-file exports from operational systems that require integration and transformation before analytical use.

#	File	Role
1	olist_orders_dataset.csv	Order header data (status, key timestamps)
2	olist_order_items_dataset.csv	Line-item level detail per order
3	olist_order_payments_dataset.csv	Payment method, instalments, value
4	olist_order_reviews_dataset.csv	Post-purchase customer review score & comments
5	olist_customers_dataset.csv	Customer identifiers and location
6	olist_products_dataset.csv	Product category and physical attributes
7	olist_sellers_dataset.csv	Seller identifiers and location
8	olist_geolocation_dataset.csv	Zip code prefix to latitude/longitude mapping
9	product_category_name_translation.csv	Maps Portuguese category names to English

Table 1: Olist dataset files

3.8 Dataset Relationships

The relationships between the core tables, including the linking keys, can be summarised as follows:

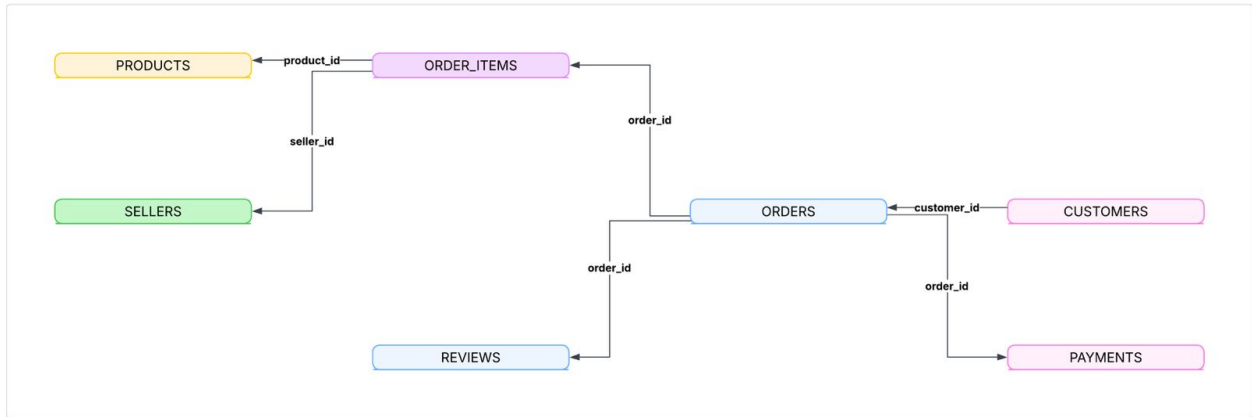


Figure 2: Operational entities of the Olist e-commerce system

This relationship structure directly informs the fact and dimension table design carried out in Task 4.

3.9 Dataset Size

The dataset contains approximately 100,000 orders placed between 2016 and 2018 across multiple marketplaces in Brazil. The extracted source files contain the following records:

Dataset	Records
Orders	99,441
Order Items	112,650
Customers	99,441
Products	32,951
Payments	103,886
Reviews	99,224
Sellers	3,095

Table 2: Record counts per dataset file

3.10 Attribute Description

A summary of key attributes per file is given below.

File	Key Attributes (Summary)
orders	order_id, customer_id, status, purchase/approved/delivered timestamps
order_items	order_id, product_id, seller_id, price, freight_value
order_payments	order_id, payment_type, instalments, payment_value
order_reviews	review_id, order_id, review_score, review timestamps
customers	customer_id, zip_code_prefix, city, state
products	product_id, category_name, weight, dimensions
sellers	seller_id, zip_code_prefix, city, state
geolocation	zip_code_prefix, latitude, longitude

Table 3: Dataset sources and attributes summary

3.11 Dataset Suitability for DW/BI Implementation

1. **Normalised relational structure** — the nine files are linked via `order_id`, `customer_id`, `product_id`, and `seller_id`, representing a normalised relational structure similar to an OLTP database.

2. **Sufficient volume for analytical processing** — approximately 100K orders (with proportionally more line items, payments, and reviews) is large enough to support meaningful aggregation, trend analysis, and OLAP operations, while remaining manageable to process end-to-end for this assignment.
3. **Rich dimensional attributes** — the dataset naturally yields at least six candidate dimensions (Customer, Product, Seller, Date, Location, and Payment), sufficient to build a well-structured dimensional data warehouse model for Task 4.
4. **Naturally supports fact and dimension table creation** — `order_items` provides transactional grain, where one row represents one order line item, corresponding to a specific product purchased within an order, with numeric measures (`price`, `freight_value`) suited to become fact table measures, while the surrounding tables cleanly separate into dimensions.
5. **Realistic, multi-faceted business scenario** — the data spans the full order lifecycle (purchase → payment → fulfilment → delivery → review), which can support the development of department-specific analytical views such as sales, customer behaviour, and operational performance.
6. **Authentic multi-source integration** — because the data arrives as nine separate files rather than one flat table, Task 2’s requirement to integrate multiple operational sources into a unified analytical environment is met organically, not artificially simulated.

4 Task 2: Data Source Identification and Preparation

4.1 Data Source Inventory

The Olist dataset contains multiple operational source files, each representing a specific business function within the e-commerce platform. Each source file must be integrated through an ETL process before loading into the data warehouse. Table 4 summarises the available data sources.

Source File	Business Function	Key Fields	Format
<code>olist_orders_dataset.csv</code>	Order Management	<code>order_id</code> , <code>customer_id</code>	CSV
<code>olist_order_items_dataset.csv</code>	Sales Transaction Processing	<code>order_id</code> , <code>product_id</code> , <code>seller_id</code>	CSV
<code>olist_customers_dataset.csv</code>	Customer Management	<code>customer_id</code>	CSV
<code>olist_products_dataset.csv</code>	Product Management	<code>product_id</code>	CSV
<code>olist_sellers_dataset.csv</code>	Seller Management	<code>seller_id</code>	CSV
<code>olist_order_payments_dataset.csv</code>	Payment Processing	<code>order_id</code>	CSV
<code>olist_order_reviews_dataset.csv</code>	Customer Feedback	<code>review_id</code> , <code>order_id</code>	CSV
<code>olist_geolocation_dataset.csv</code>	Location Management	<code>zip_code_prefix</code>	CSV
<code>product_category_name_translation.csv</code>	Category Translation	<code>category_name</code>	CSV

Table 4: Olist data source inventory

4.2 Source System Description

Each operational source file contributes different information required for analytical processing.

4.2.1 Orders Source

File: `olist_orders_dataset.csv`

Purpose: Stores order-level transaction information, including status and key lifecycle timestamps.

Attribute	Purpose
order_id	Unique order identifier
customer_id	Links order to customer
order_status	Current order state
order_purchase_timestamp	Order creation time
order_delivered_customer_date	Delivery completion date

Table 5: Key attributes: Orders source

Analytical use: Supports order volume analysis, delivery performance analysis, and Date dimension creation.

4.2.2 Order Items Source

File: olist_order_items_dataset.csv

Purpose: Stores detailed product-level information within each order. This is the most important source file for the fact table, as it defines the transaction grain.

Attribute	Purpose
order_id	Transaction reference
product_id	Purchased product
seller_id	Product supplier
price	Product selling price
freight_value	Shipping cost

Table 6: Key attributes: Order Items source

Analytical use: Provides sales measures, defines the transaction grain, and establishes product and seller relationships.

4.2.3 Customer Source

File: olist_customers_dataset.csv

Purpose: Stores customer identification and location information (customer_id, city, state, zip_code_prefix).

Analytical use: Forms the basis of the Customer dimension (Dim_Customer).

4.2.4 Product Source

File: olist_products_dataset.csv

Purpose: Stores product characteristics (product_id, category, weight, dimensions).

Analytical use: Forms the basis of the Product dimension (Dim_Product).

4.2.5 Seller Source

File: olist_sellers_dataset.csv

Purpose: Stores marketplace seller identification and location information.

Analytical use: Forms the basis of the Seller dimension (Dim_Seller).

4.2.6 Payment Source

File: olist_order_payments_dataset.csv

Purpose: Stores payment behaviour (payment_type, installments, payment_value).

Analytical use: Supports payment behaviour analysis and forms the basis of a Payment dimension.

4.2.7 Review Source

File: olist_order_reviews_dataset.csv

Purpose: Stores customer satisfaction information (`review_score`, `review_comment`, `review_date`).

Analytical use: Supports customer satisfaction analysis and quality metrics.

4.3 Source Integration Relationships

The integration process depends on common keys shared between operational source files. Table 7 summarises the primary integration keys and the relationships they establish.

Key	Relationship
customer_id	Customers → Orders
order_id	Orders → Order Items, Payments, Reviews
product_id	Order Items → Products
seller_id	Order Items → Sellers
zip_code_prefix	Location information (Customers/Sellers → Geolocation)

Table 7: Source integration keys

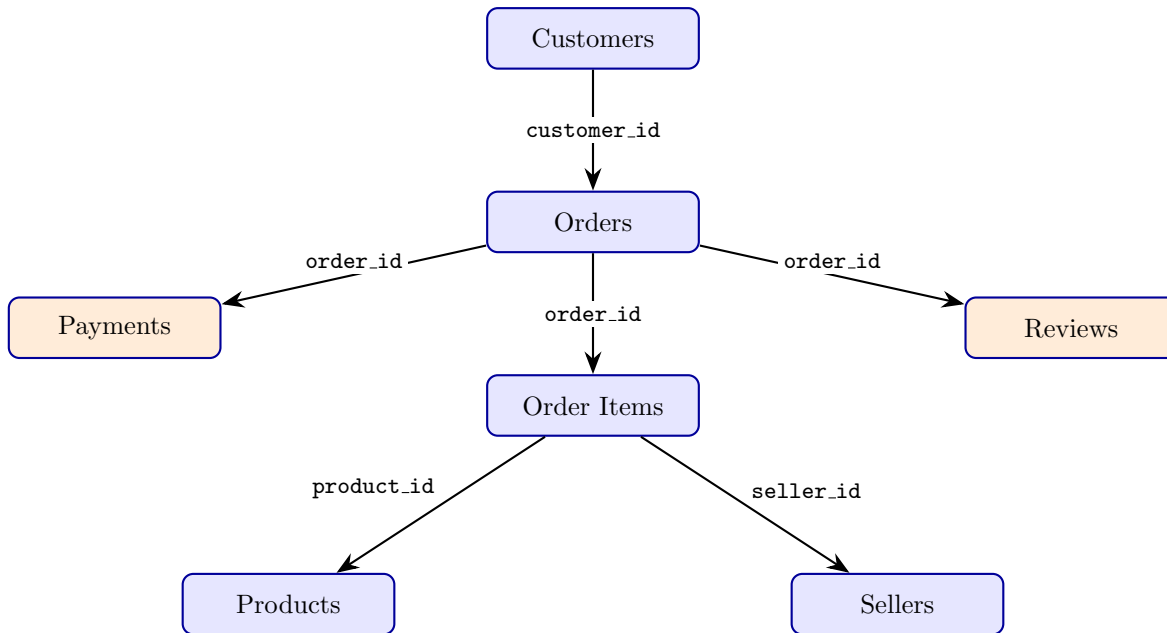


Figure 3: Source integration keys between the Olist tables

4.4 Data Quality Assessment

Before loading data into the warehouse, the source files must be evaluated for data quality issues that could otherwise propagate into the dimensional model.

4.4.1 Missing Values

Some products have missing values in `product_category_name` (610 products, confirmed during staging validation in Section 4.9). Missing categories will be replaced with a placeholder value of "Unknown" so that affected records are not excluded from analysis.

4.4.2 Duplicate Records

Key identifier fields must be checked for uniqueness prior to loading:

- `order_id` in the Orders source is expected to be unique per order.
- `customer_id` in the Customers source is expected to be unique per customer.

4.4.3 Inconsistent Categories

Product categories are stored in Portuguese (e.g. `beleza_saude`). The `product_category_name_translation.csv` lookup file will be used to translate these into English equivalents (e.g. *Health & Beauty*) for readability in reports and dashboards. Exploratory data analysis confirmed that, even after applying this lookup, 13 products (out of 32,341 products with a known category) have a category name with no matching English translation. These records will retain their original Portuguese category name as a fallback, rather than being dropped, so that they remain available for quantitative analysis even where the category label is not translated.

4.4.4 Data Type Issues

Timestamp fields such as `order_purchase_timestamp` are stored as text strings in the raw CSV files and must be converted to proper DATETIME types before being used in date-based calculations or linked to the Date dimension.

4.4.5 Incomplete Delivery Information

Orders that are cancelled or otherwise unavailable may not contain delivery completion timestamps. These records should not be treated as data errors; instead, delivery-related measures (e.g. delivery duration, on-time delivery rate) should be calculated only for orders with a completed delivery status, so that incomplete orders do not distort delivery performance metrics.

4.4.6 Geolocation Duplicate Records

Exploratory data analysis identified 261,831 duplicate rows within the `olist_geolocation_dataset.csv` source file. This occurs because a single `zip_code_prefix` can legitimately map to multiple near-identical latitude/longitude readings, as one postal prefix covers a wide geographic area. If loaded without deduplication, this would cause the Location dimension to contain far more rows than the number of genuinely distinct locations, inflating join cardinality during ETL. This source file will therefore be deduplicated by `zip_code_prefix` (retaining a single representative latitude/longitude pair per prefix) before being used to construct the Location dimension in Task 4.

4.4.7 Inconsistent Seller City Values

Profiling of the staging tables (Section 4.9) showed that some values in the seller `city` field also contain state and country text, for example *rio de janeiro, rio de janeiro, brasil*. These values will be cleaned and standardised during transformation so that seller locations can be grouped and joined consistently.

4.4.8 Outlier Presence in Numeric Measures

An IQR-based outlier analysis was performed on the key numeric measures identified as candidate fact table fields. Table 8 summarises the findings.

Source	Column	Outlier %
Order Items	price	7.48%
Order Items	freight_value	10.77%
Payments	payment_value	7.68%
Products	product_weight_g	13.81%

Table 8: IQR-based outlier percentages for key numeric measures

These outlier rates are non-trivial but expected in a genuine retail transaction dataset, where a small number of bulk or high-value orders naturally produce right-skewed price and freight distributions. Rather than removing these records, they will be retained in the warehouse and flagged where necessary, since they represent legitimate business activity (e.g. premium products, large or heavy shipments) rather than data entry errors.

4.5 Data Cleaning and Transformation Requirements

Table 9 summarises the transformation requirements identified from the data quality assessment above.

Issue	Transformation
Missing categories	Replace with "Unknown"
Portuguese category names	Translate to English using lookup file; retain original name where no translation match exists
Date strings	Convert to proper date/datetime format
Duplicate records (business keys)	Identify and remove duplicates
Duplicate geolocation records	Deduplicate by zip_code_prefix, retaining one representative coordinate pair
Inconsistent seller city text	Standardise city names (remove embedded state/country text)
Null delivery dates	Exclude from delivery metrics; retain record otherwise
Multiple payment records per order	Consolidate payment records at the required analytical grain, while preserving individual payment attributes (type, instalments, value) for detailed analysis
Outliers in price, freight, payment, weight	Retain and flag rather than remove, as they reflect genuine high-value/bulk transactions

Table 9: Data cleaning and transformation requirements

4.6 Staging Area Design

Before loading into the warehouse, extracted data will pass through a dedicated ETL pipeline consisting of extraction, staging, and transformation stages ahead of the Enterprise Data Warehouse.

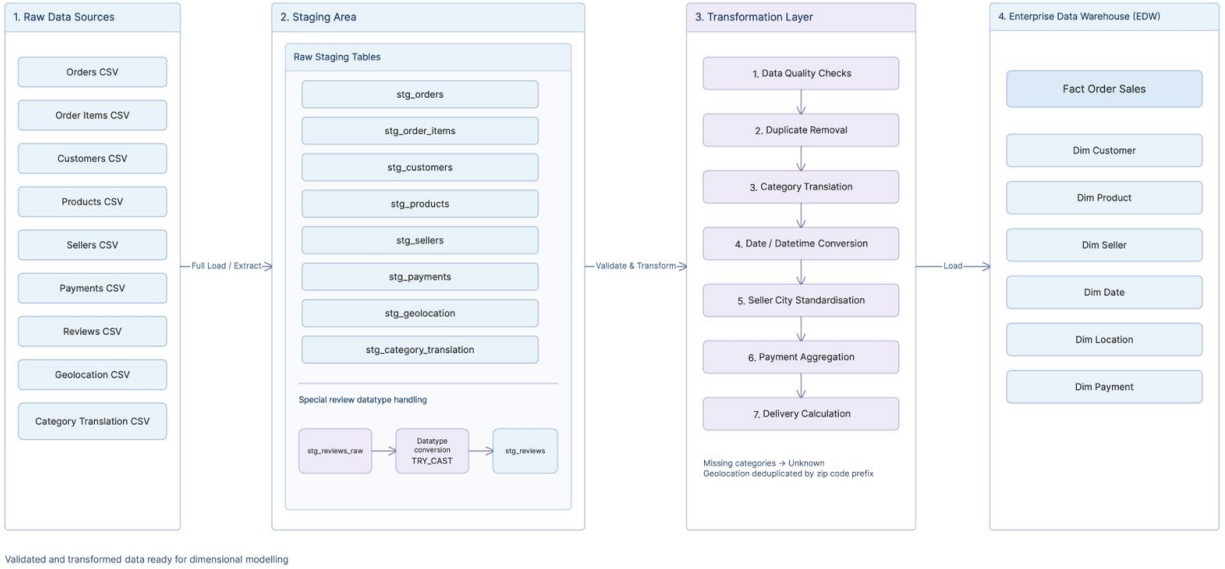


Figure 4: Operational entities of the Olist e-commerce system

The staging area serves the following purposes:

- Provides temporary storage for extracted files prior to transformation.
- Enables data quality checking in isolation from the source systems.
- Hosts transformation processing (cleaning, standardisation, translation).
- Maintains a clear separation between operational source systems and the warehouse, so that raw extracts can be re-processed if a transformation error is identified.

Table 10 lists the proposed staging tables and their corresponding source files, covering all nine source files identified in the Data Source Inventory. One additional intermediate table, `stg_reviews_raw`, is used as a buffer for the review file (see Section 4.8).

Staging Table	Source File
stg_orders	olist_orders_dataset.csv
stg_order_items	olist_order_items_dataset.csv
stg_customers	olist_customers_dataset.csv
stg_products	olist_products_dataset.csv
stg_sellers	olist_sellers_dataset.csv
stg_payments	olist_order_payments_dataset.csv
stg_reviews	olist_order_reviews_dataset.csv
stg_reviews_raw (intermediate)	olist_order_reviews_dataset.csv
stg_geolocation	olist_geolocation_dataset.csv
stg_category_translation	product_category_name_translation.csv

Table 10: Proposed staging tables

4.7 Source-to-Staging Mapping

The ETL extraction phase loads each operational CSV file into a corresponding staging table without altering the original business meaning. The staging layer provides a controlled environment where data quality checks

and transformations can be performed before loading into the dimensional warehouse. Since the Olist dataset is a static, historical extract rather than a live operational feed, a full load strategy is used for all source files; an incremental load strategy would be more appropriate in a production environment where the source system is continuously updated. Table 11 defines the mapping between each source file and its corresponding staging table.

Source File	Staging Table	Loading Method
olist_orders_dataset.csv	stg_orders	Full load
olist_order_items_dataset.csv	stg_order_items	Full load
olist_customers_dataset.csv	stg_customers	Full load
olist_products_dataset.csv	stg_products	Full load
olist_sellers_dataset.csv	stg_sellers	Full load
olist_order_payments_dataset.csv	stg_payments	Full load
olist_order_reviews_dataset.csv	stg_reviews	Full load (via stg_reviews_raw)
olist_geolocation_dataset.csv	stg_geolocation	Full load
product_category_name_translation.csv	stg_category_translation	Full load

Table 11: Source-to-staging mapping and loading method

Each staging table retains the same column names and grain as its source file at this stage; renaming, type conversion, and business-rule transformations (as identified in Section 4.5) are applied in the subsequent transformation phase, not during extraction. This separation ensures that the staging layer always reflects an accurate, traceable copy of the original source data, which is useful for auditing and re-processing if a transformation error is later identified. The review file is the one exception: because of its multiline free-text fields, it is first loaded into an intermediate raw table and then converted into **stg_reviews** (see Section 4.8).

4.8 Data Preparation

4.8.1 Handling Complex Review Text Data

Most source files were loaded directly into their staging tables. The review file, `olist_order_reviews_dataset.csv`, could not be loaded this way because of the structure of its free-text fields.

Problem: direct loading failed. The initial attempt to bulk load the file straight into `Staging.stg_reviews` failed with the following errors:

- *Data conversion failed while converting column review_creation_date*
- *Bulk load data conversion error (type mismatch or invalid character)*
- *Potential loss of data*

Root cause. The problem was not the data values but the structure of the CSV file. The columns `review_comment_title` and `review_comment_message` contain customer-written text, which may include commas, quotation marks, and line breaks within a single quoted value, for example:

```
"Produto excelente,
entrega rapida,
muito satisfeito"
```

A plain delimiter-based load interprets these embedded commas and line breaks as column and row boundaries. This shifted values into the wrong columns (for example, datetime values landing in text columns), which caused the conversion failures.

Solution: raw staging layer. An intermediate table, `Staging.stg_reviews_raw`, was introduced to preserve the original CSV structure. All columns were defined as text (`NVARCHAR`) so that no datatype conversion is attempted during extraction. This is consistent with the principle that the staging layer should preserve source data before any business rules are applied. Figure 5 shows the table definition.

```

52
53 DROP TABLE IF EXISTS Staging.stg_reviews_raw;
54 GO
55
56 CREATE TABLE Staging.stg_reviews_raw
57 (
58     review_id          NVARCHAR(100),
59     order_id           NVARCHAR(100),
60     review_score       NVARCHAR(50),
61     review_comment_title NVARCHAR(MAX),
62     review_comment_message NVARCHAR(MAX),
63     review_creation_date NVARCHAR(100),
64     review_answer_timestamp NVARCHAR(100)
65 );
66 GO

```

Figure 5: Creation of intermediate raw review staging table

The file was then loaded using `BULK INSERT` with CSV-aware options, summarised in Table 12.

Option	Purpose
<code>FORMAT = 'CSV'</code>	Enables SQL Server to apply CSV quoting rules
<code>FIELDQUOTE = '"'</code>	Keeps commas inside quoted review text within a single field
<code>ROWTERMINATOR = '0x0d0a'</code>	Treats only a carriage return and line feed as the end of a record, so line breaks inside quoted text are preserved
<code>CODEPAGE = '65001'</code>	UTF-8 encoding, required for Portuguese characters
<code>FIRSTROW = 2</code>	Skips the CSV header row

Table 12: BULK INSERT options used for the review file

Validation of the raw load. After loading, the row count of the raw table was compared against the source file. The result of 99,224 rows matches the record count in Table 2, confirming that no records were split or lost (Figure 6). A sample of the raw records (Figure 7) confirmed that identifiers and comment text were loaded correctly and that timestamps were preserved as text.

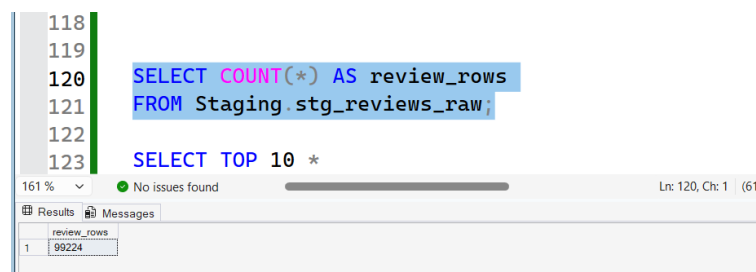


Figure 6: Validation of raw review record count

123
124
125
126

```
SELECT TOP 10 *
FROM Staging.stg_reviews_raw;
```

214 % No issues found Lr: 123, Ch: 1 (46 chars, 2 lines) SPC CRLF UTF-

	review_id	order_id	review_score	review_comment_title	review_comment_message	review_creation_date	review_answer_timestamp
1	7bc2406110b926393aa56f0a40eba40	73fc7af87114b39712e6da79b0a377eb	4	NULL	NULL	2018-01-18 00:00:00	2018-01-18 21:46:59
2	80e641a11e56f04c1ad469d5645fdfe	a548910a1c6147796b98df73dbeba33	5	NULL	NULL	2018-03-10 00:00:00	2018-03-11 03:05:13
3	228ce5500dc1d8e020d8d1322874b6f0	f9e4b658b201a9f2ecdecbb34bed034b	5	NULL	NULL	2018-02-17 00:00:00	2018-02-18 14:36:24
4	e64fb393e7b32834bb789f8bb30750e	658677c97b385a9be170737859d3511b	5	NULL	Recebi bem antes do prazo estipulado.	2017-04-21 00:00:00	2017-04-21 22:02:06
5	f7c4243c7fe1938f181bec41a392bdeb	8e6fb81e283fa7e4f11123a3fb894f1	5	NULL	Parabéns lojas lannister adorei comprar pela Int...	2018-03-01 00:00:00	2018-03-02 10:26:53
6	15197aa66f4d0650b5434f1b46cda19	b18dcd73be66366873cd26c5724d1dc	1	NULL	NULL	2018-04-13 00:00:00	2018-04-16 00:39:37
7	079bbee5d1b850860defd761afa7f16	e48aa0d2dec3a2e87348811bdcfd22b	5	NULL	NULL	2017-07-16 00:00:00	2017-07-18 19:30:34
8	7c6400515c67679fbee952a7525281ef	c31a859e34e3adac22f376954e19b39d	5	NULL	NULL	2018-08-14 00:00:00	2018-08-14 21:36:06
9	a36f7f6433de0aefb97da197c554c	9c214ac970e84273583ab523dfaf09b	5	NULL	NULL	2017-05-17 00:00:00	2017-05-18 12:05:37
10	8670d52e15e00043ae7de4c01cc2fe06	b9bf720beb4ab372876008589c62129	4	recomendo	aparelho eficiente. no site a marca do aparelho ...	2018-05-22 00:00:00	2018-05-23 16:45:47

Figure 7: Sample records from raw review staging table

Transformation into the cleaned staging table. Once the raw data was verified, it was moved into `Staging.stg_reviews` with `TRY_CAST` validation applied:

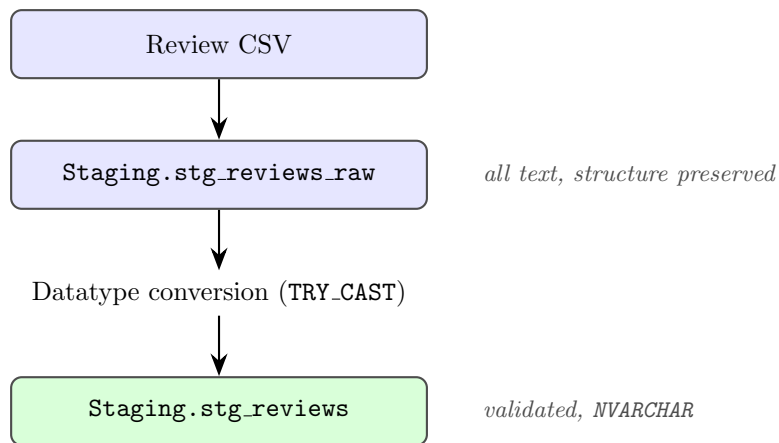


Figure 8: Review data flow from source CSV to cleaned staging table

- `review_score` was validated as an integer with `TRY_CAST`; values that are not valid integers become `NULL`.
- `review_creation_date` and `review_answer_timestamp` were validated as `DATETIME` values in the same way.
- Review comment fields were carried across unchanged.

Consistent with the all-`NVARCHAR` staging design, the final conversion to `INT` and `DATETIME` is applied in the fact load (Task 5).

`TRY_CAST()` was used instead of `CAST()`. With `CAST()`, a single malformed value aborts the entire load. `TRY_CAST()` returns `NULL` for the invalid value and lets the process continue, which is safer in an ETL pipeline, and the affected rows can then be identified and reviewed. The final row count of `stg_reviews` was 99,224, identical to the raw table, and the sample in Figure 9 confirms that the values were carried across correctly.

206
207
208

```
SELECT TOP 10 *
FROM Staging.stg_reviews;
```

214 % No issues found Ln: 207, Ch: 1 (42 chars, 2 lines) SPC CRLF UTF-8

Results Messages

	review_id	order_id	review_score	review_comment_title	review_comment_message	review_creation_date	review_answer_timestamp
1	7bc2406110b926393aa5080a40eba40	73fc7af87114b39712e6da79b0a377eb	4	NULL	NULL	Jan 18 2018 12:00AM	Jan 18 2018 9:46PM
2	80e641a11e5904c1ad469d5645dfdde	a548910a1c6147796b98fd73dbeba33	5	NULL	NULL	Mar 10 2018 12:00AM	Mar 11 2018 3:05AM
3	228ce5500dc1d8e020d8d1322874b6f0	f9e4b658b201a9f2ecdecbb34bed034b	5	NULL	NULL	Feb 17 2018 12:00AM	Feb 18 2018 2:36PM
4	e64fb393e7b32834bb789f8bb30750e	658677c97b385a9be170737859d3511b	5	NULL	Recebi bem antes do prazo estipulado.	Apr 21 2017 12:00AM	Apr 21 2017 10:02PM
5	f7c4243c7fe1938f181bec41a392bdeb	8e6bfb81e283fa7e4f11123a3fb894f1	5	NULL	Parabéns lojas lannister adorei comprar pela Int...	Mar 1 2018 12:00AM	Mar 2 2018 10:26AM
6	15197aa69f4d0650b5434f1b46cda19	b18dcd73be66366873cd26c5724d1dc	1	NULL	NULL	Apr 13 2018 12:00AM	Apr 16 2018 12:39AM
7	079bbe5d1b850860defdf761afa7f16	e48aa0d2dccc3a2e87348811bcdff22b	5	NULL	NULL	Jul 16 2017 12:00AM	Jul 18 2017 7:30PM
8	7c6400515c67679fbee952a7525281ef	c31a859e34e3adac22f376954e19b39d	5	NULL	NULL	Aug 14 2018 12:00AM	Aug 14 2018 9:36PM
9	a3bf769433de0aefb97da197c554c	9c214ac970e84273583ab523dfafd09b	5	NULL	NULL	May 17 2017 12:00AM	May 18 2017 12:05PM
10	8670d52e15e00043ae7de4c01cc2fe06	b9bf720beb4ab3728760088589c62129	4	recomendo	aparelho eficiente. no site a marca do aparelho ...	May 22 2018 12:00AM	May 23 2018 4:45PM

Figure 9: Review staging table after TRY_CAST validation

Role of the raw table. `stg_reviews_raw` is an ETL buffer, not a warehouse table. It exists only to safely ingest complex source data, and the dimensional model will read from the cleaned `stg_reviews` table. The resulting flow for reviews is:

Source CSV -> Raw Staging -> Cleaned Staging -> Dimension/Fact -> BI

4.9 Staging Validation

After all nine source files were loaded, a combined validation script was run in SSMS to confirm that the staging layer accurately reflects the source data before any transformation is applied. Figure 10 shows the results. The results grid is scrollable; the ninth row (`stg_sellers`) is not visible in the screenshot but also returned PASS.

The screenshot shows a SQL IDE with a query window and a results pane. The query window displays a SQL script for validating staging tables. The results pane shows the execution results, including row counts, bad rows, and orphan rows for various staging tables.

table_name	actual_rows	expected_rows	result
stg_category_translation	71	71	PASS
stg_customers	99441	99441	PASS
stg_geolocation	1000163	1000163	PASS
stg_order_items	112650	112650	PASS
stg_orders	99441	99441	PASS
stg_payments	103886	103886	PASS
stg_products	32951	32951	PASS
stg_reviews	99224	99224	PASS

check_name	bad_rows
orders keys	0
order_items keys	0
customers keys	0
sellers keys	0
products keys	0
reviews keys	0

check_name	orphan_rows
orders without customer	0
order_items without order	0
order_items without pro...	0
order_items without seller	0
payments without order	0
reviews without order	0

seller_city	seller_state
novo hamburgo, rio grande do sul, brasil	RS
rio de janeiro, rio de janeiro, brasil	RJ

blank_delivered_customer_date	blank_product_category
2965	610

Figure 10: Staging validation results across all nine staging tables

Four categories of checks were performed, as summarised in Table 13.

Check	Description	Result
Row count reconciliation	Row count of each staging table compared with the source file count	All PASS
Key integrity	Null or blank primary/business keys in orders, order items, customers, sellers, products, and reviews	0 bad rows
Referential integrity	Orphan records between related tables (e.g. orders without a customer, order items without a product or seller)	0 orphan rows
Data quality profiling	Blank delivery dates, blank product categories, and inconsistent seller city values	See below

Table 13: Staging validation checks and outcomes

The profiling checks identified the following issues, which are handled in the transformation phase:

- **2,965 orders** have no delivery completion date. As described in Section 4.4, these are retained but excluded from delivery metrics.
- **610 products** have a blank category name. These are replaced with "Unknown".
- **Inconsistent seller city values** were found, where the city field also contains state and country text (e.g. *rio de janeiro, rio de janeiro, brasil*). These values will be cleaned and standardised during transformation.

4.10 Task 2 Outcome

Following the data source identification, quality assessment, staging design, and staging validation described above, the raw Olist operational source files are transformed into validated and structured data, ready to support warehouse architecture design and dimensional modelling in the following tasks:

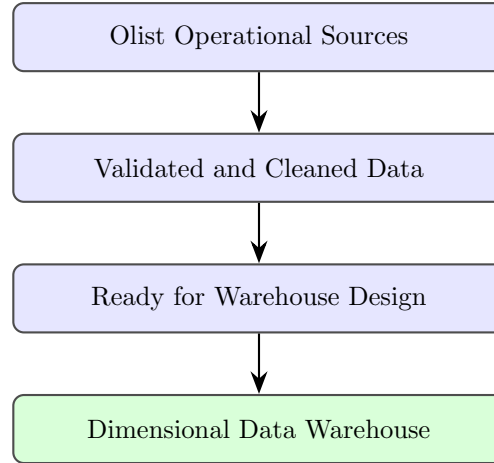


Figure 11: Task 2 outcome: from operational sources to the dimensional warehouse

The identified source structures, integration keys, and transformation requirements provide the foundation for designing the warehouse architecture and dimensional model in the following tasks.

5 Task 3: Data Warehouse Architecture Design

5.1 Architecture Overview

The proposed architecture follows a four-layer Data Warehouse and Business Intelligence design, moving data from raw operational sources through a staging and transformation pipeline into a dimensional Enterprise Data Warehouse, before being served to end users through OLAP tools and BI dashboards. Figure 12 presents the complete architecture.

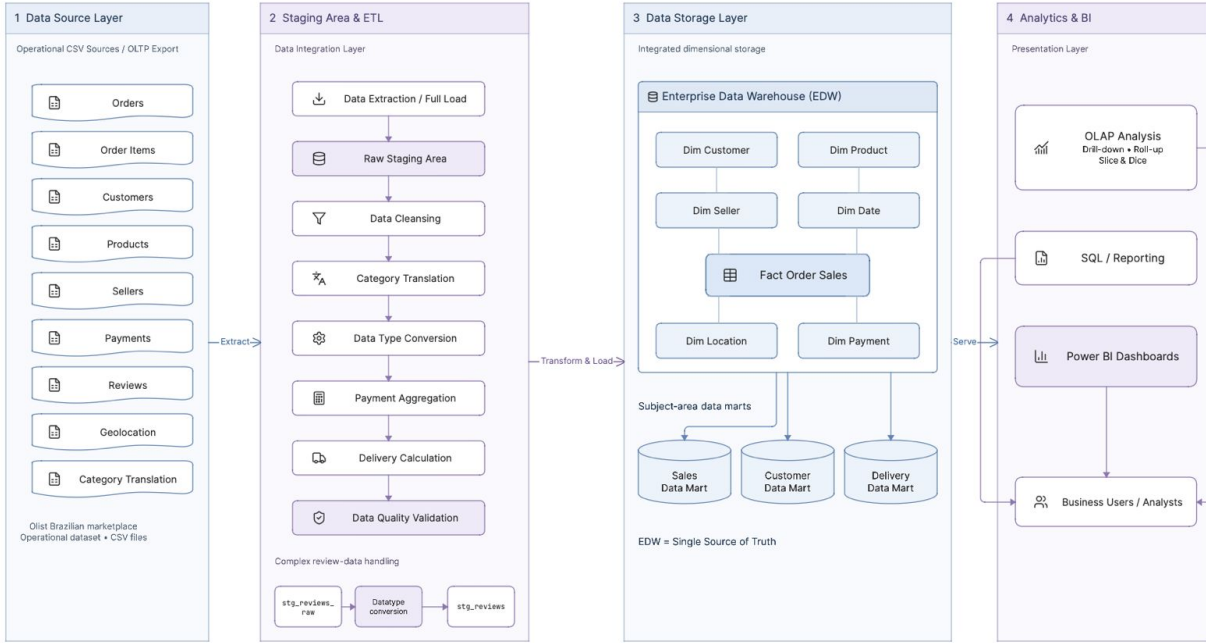


Figure 6. Olist data warehouse and business intelligence architecture.

Figure 12: Horizontal DW/BI architecture for the Olist e-commerce data warehouse

5.2 Data Source Layer

The Data Source Layer consists of the nine operational CSV files identified in Task 1 and Task 2 (`orders.csv`, `order_items.csv`, `customers.csv`, `products.csv`, `sellers.csv`, `payments.csv`, `reviews.csv`, `geolocation.csv`, and `category_translation.csv`), representing the underlying OLTP system of the Olist marketplace. These nine relational tables represent exported operational data from Olist’s marketplace system and capture individual business transactions across order management, customer management, product management, payment processing, and delivery management.

This layer is characterised by:

- **Heterogeneous structure** — nine separate flat files, each corresponding to a distinct operational entity.
- **Transactional grain** — data is recorded at the individual transaction level (e.g. one row per order line item), not pre-aggregated.
- **No analytical optimisation** — the source files are exports from a system designed for recording transactions, not for supporting multi-dimensional queries, which is why the subsequent layers are required.

5.3 Staging Area and ETL (Data Integration Layer)

The Staging Area and ETL pipeline together form the Data Integration Layer, responsible for extracting, cleansing, and preparing source data before it is loaded into the warehouse.

5.3.1 Staging Area (Raw Buffer)

Extracted data is first written into a set of raw staging tables (`stg_orders`, `stg_customers`, `stg_products`, etc., as defined in Section 4.6). The staging area isolates raw ingested data from the operational source

and provides a controlled environment for validation, cleansing, and transformation before loading into the warehouse. Complex source files such as the review data use an additional intermediate raw table (`stg_reviews_raw`, Section 4.8) to preserve the original file structure before datatype conversion.

5.3.2 ETL Processing Pipeline

The transformation pipeline applies the following ordered processing steps:

1. **Extract and Ingest** — a full bulk load strategy is used for CSV ingestion, consistent with the static, historical nature of the dataset (Section 4.7).
2. **Data Cleansing** — duplicate geolocation zip-code coordinates are removed, as identified during EDA (261,831 duplicate rows; Section 4.4).
3. **Category Translation** — Portuguese product category names are mapped to English strings using `category_translation.csv`, with the original name retained where no translation match exists.
4. **Type Casting and Parsing** — timestamp strings (e.g. `order_purchase_timestamp`) are converted to proper `TIMESTAMP` types.
5. **Payment Aggregation** — payment records are transformed according to the analytical grain required by the warehouse, while preserving individual payment attributes (type, instalments, value) needed for detailed analysis.
6. **Delivery Calculation** — estimated versus actual delivery variance is derived for completed orders only, excluding orders with incomplete delivery information (Section 4.4).

5.3.3 Enforced Data Quality Rules

The following data quality rules, established in Task 2, are enforced during the ETL process:

- Missing product categories default to "Unknown".
- Geolocation records are deduplicated by zip code prefix.
- Seller city values are standardised.
- Order timestamp formats are standardised.
- Payment and freight totals are reconciled against order-level records.

5.4 Data Storage Layer

The Data Storage Layer holds the cleaned, integrated, and conformed data, and is organised into the Enterprise Data Warehouse and its subject-area data marts.

5.4.1 Enterprise Data Warehouse (Star Schema)

The EDW is structured as a star schema, centred on a single fact table, `Fact_Order_Sales`, which stores sales transaction measures such as quantity, price, and freight value, with additional derived measures from related order-level information such as delivery duration and customer satisfaction indicators where applicable. This fact table is surrounded by six dimension tables:

Dimension	Key Attributes
Dim_Customer	Customer city, state, zip
Dim_Product	Product category, dimensions, weight
Dim_Seller	Seller location and region
Dim_Date	Year, quarter, month, day
Dim_Location	Latitude/longitude zip geolocation mapping
Dim_Payment	Payment type and instalments

Table 14: Dimension tables surrounding Fact_Order_Sales

Dim_Location is maintained separately from Dim_Customer and Dim_Seller because the geographic enrichment sourced from `olist_geolocation_dataset.csv` (coordinates and regional mapping derived from `zip_code_prefix`) provides analytical attributes that are independent from customer and seller master data, and can be reused for both customer-side and seller-side geographic analysis (e.g. delivery distance) without duplicating coordinate data inside both dimensions.

The full grain, measure, and key definitions for this schema are detailed in Task 4.

5.4.2 Subject Area Data Marts

Three department-specific data marts are proposed, each serving a distinct analytical audience. The Sales Data Mart is implemented in Task 6, and the Customer and Delivery marts are proposed extensions:

- **Sales Data Mart** — sales volume, revenue trends, and top product categories.
- **Customer Data Mart** — customer retention, geographic distribution, and review sentiment.
- **Delivery Data Mart** — shipping performance, carrier delay analysis, and freight cost.

The EDW acts as the single source of truth: it holds the standardised dimensional model, while each data mart serves department-specific analysts without duplicating the underlying transformation logic. The implemented Sales Data Mart is detailed in Task 6.

5.5 Analytics and BI (Presentation Layer)

The Presentation Layer is the point at which business users interact with the warehouse’s data, consisting of three complementary components:

5.5.1 Reporting Tools

Ad-hoc SQL queries and operational reports support use cases such as daily order fulfilment status logs, seller payout summaries and commission metrics, and customer support review-trigger alerts.

5.5.2 OLAP Tools (Multidimensional)

Multidimensional cube slicing and dicing enables analysts to explore the warehouse without writing SQL directly, including:

- **Drill-down:** Year → Quarter → Month → Day
- **Roll-up:** City → State → Region Sales
- **Slice and dice:** Product Category vs. Payment Type

5.5.3 Power BI Dashboards

Power BI dashboards provide interactive visual analysis of business performance indicators, including sales trends, customer behaviour, logistics performance, and seller analysis. The detailed dashboard pages and their specific visual components are presented in Task 7.

5.6 Summary of Layer Interactions

Table 15 summarises how data flows through the architecture and which task documents each stage in detail.

Layer	Function	Detailed in
Data Source Layer	Provides raw operational data	Task 1, Task 2
Staging Area & ETL	Extracts, cleans, and transforms data	Task 2, Task 5
Data Storage Layer	Warehouses and marts the cleaned data	Task 4, Task 6
Analytics and BI	Delivers OLAP analysis and dashboards	Task 7, Task 8

Table 15: Summary of architecture layers and corresponding tasks

This architecture ensures a clear separation of concerns: the operational OLTP system remains untouched by analytical workloads, data quality is enforced within the staging and transformation pipeline before data reaches the warehouse, and business users interact only with a purpose-built, query-optimised presentation layer. The Enterprise Data Warehouse serves as the single source of truth, empowering executive, marketing, and logistics teams with interactive analytical decision support and self-service data discovery.

6 Task 4: Dimensional Data Warehouse Design and Implementation

6.1 Business Process

The fact table models product-level sales transactions across the Olist marketplace — the order fulfilment process, from purchase through to delivery and customer review.

6.2 Fact Table Grain

One row in `Fact_Order_Sales` represents **one distinct product, purchased from one seller, within one order**.

This grain was chosen over the raw source grain (one row per source order-item record) after verifying the underlying data: the `order_items` source file contains multiple order-item records for the same order-product-seller combination whenever a product is purchased more than once within an order. Profiling confirmed that `price` and `freight_value` are constant within every (order, product, seller) group — 7,088 of 102,425 such groups contain more than one record, and in every case the per-unit price and freight value do not vary within the group. Aggregating to one row per (order, product, seller) with an explicit `quantity` measure is therefore lossless, and produces a more meaningful grain than a source-level record where quantity would trivially always equal 1.

The aggregation logic described here is implemented during the ETL phase (Task 5) by grouping source order item records by `order_id`, `product_id` and `seller_id`, and deriving `quantity` as the number of source rows represented by each warehouse fact row (`COUNT(*)`).

6.3 The Grain-Mismatch Problem and How It Is Resolved

As flagged during the architecture design (Task 3), three related source tables do not share the fact table's grain:

- `order_payments` and `order_reviews` are recorded at **order** grain, not line-item grain.
- A single order can have multiple payment records (2,961 of 99,440 orders have more than one payment record, e.g. split across instalments or payment methods).

Rather than silently duplicating order-level facts across every line item of a multi-line order (which risks double-counting if a BI tool naively sums them), two design decisions were made:

1. `order_id` is retained on the fact table as a **degenerate dimension** — a dimension attribute with no separate dimension table — specifically so that BI tools can de-duplicate order-level measures (e.g. take `DISTINCT order_id` before averaging `review_score`) rather than over-counting them.
2. For orders with multiple payment records, `payment_key` stores the **primary payment type**, selected as the payment record with the highest `payment_value` for that order. `order_total_payment_value` and `order_total_installments` are summed across *all* payment records belonging to the order. Complete payment transaction detail remains available in the staging layer and can support a future payment transaction fact table if required.

Placement of Review Score

Although reviews represent customer feedback events, `review_score` is included in `Fact_Order_Sales` because each review is associated with an order and is analysed primarily as a customer satisfaction indicator related to completed purchases.

6.4 Measures

Table 16 summarises the measures carried on the fact table.

Measure	Grain	Description
quantity	Line	Number of source order-item records represented by this fact row (derived as <code>COUNT(*)</code> during ETL)
unit_price	Line	Selling price per individual product unit, before aggregation
total_price	Line	Aggregated selling value calculated as $\text{quantity} \times \text{unit_price}$
unit_freight_value	Line	Shipping cost per individual product unit, before aggregation
total_freight_value	Line	Aggregated shipping cost calculated as $\text{quantity} \times \text{unit_freight_value}$
order_total_payment_value	Order (replicated)	Sum of all payment records for the order
order_total_installments	Order (replicated)	Sum of instalment counts across the order's payments
delivery_duration_days	Order (replicated)	Days between purchase and delivery; NULL if not yet delivered
is_delivered_late	Order (replicated)	Whether delivery occurred after the estimated date; NULL if not yet delivered
review_score	Order (replicated)	Customer satisfaction score (1–5); NULL if no review submitted

Table 16: Fact_Order_Sales measures

6.5 Dimension Tables and Foreign Keys

Table 17 lists the foreign keys on the fact table and the dimension each one references.

Foreign Key	References	Purpose
order_purchase_date_key	Dim_Date	When the order was placed
customer_key	Dim_Customer	Who bought
customer_location_key	Dim_Location	Customer's geographic coordinates (role-playing)
product_key	Dim_Product	What was bought
seller_key	Dim_Seller	Who sold it
seller_location_key	Dim_Location	Seller's geographic coordinates (role-playing)
payment_key	Dim_Payment	Primary payment type for the order

Table 17: Fact_Order_Sales foreign keys

Dim_Location is a **role-playing dimension**: it is referenced twice by the fact table, once as the customer's location and once as the seller's location. This is consistent with the justification given in Task 3 for keeping it as an independent, reusable dimension rather than embedding coordinates separately inside Dim_Customer and Dim_Seller.

Future extensions may include additional role-playing date keys (e.g. delivery date and estimated delivery date) for detailed delivery milestone analysis. In the current design, delivery performance is captured through the `delivery_duration_days` and `is_delivered_late` measures.

6.6 Star Schema Diagram

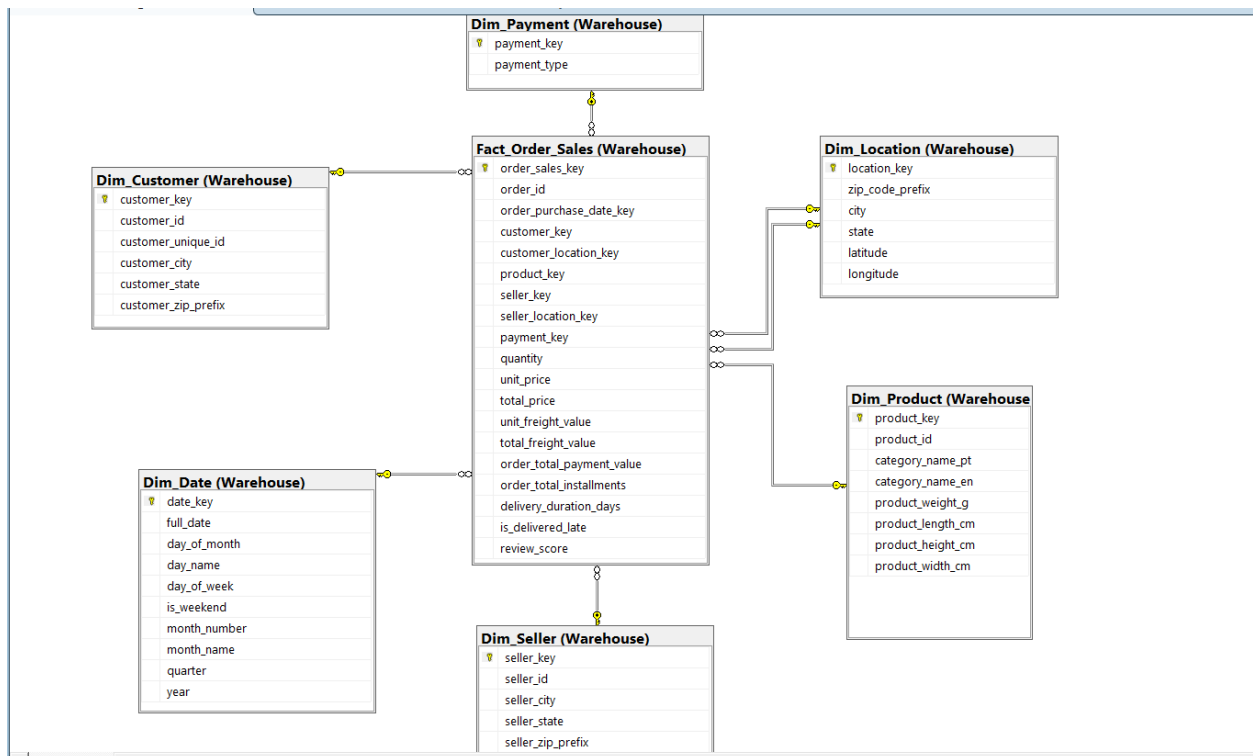


Figure 13: Star schema for Fact_Order_Sales and its six dimension tables (Dim_Location shown as a role-playing dimension)

6.7 Design Assumptions

1. **Aggregated line grain.** The fact table aggregates the source `order_items` records to one row per (order, product, seller), with `quantity` as an explicit measure, rather than keeping the source's one-row-per-order-item-record grain (verified lossless; see Section 6.2).
2. **Order-level measures are replicated, not summed blindly.** `review_score`, `delivery_duration_days`, `is_delivered_late`, and the order-level payment totals are attached to every line of a multi-line order. The degenerate `order_id` dimension exists specifically so these can be correctly de-duplicated during analysis.
3. **Primary payment type simplification.** Where an order has multiple payment records, the fact table's `payment_key` reflects only the primary payment type (highest payment value). The full multi-payment detail is preserved in the staging layer and can support a future payment transaction fact table if payment-method-level analysis is required.
4. **Unknown-member rows.** Every dimension table includes a row at surrogate key `-1` representing an unknown or missing lookup value, so that a fact row can always be linked to a valid dimension key even when source data is incomplete (e.g. a product with no recorded category).
5. **Dim.Date is pre-populated independently of the ETL load,** covering the full calendar range of the dataset, rather than being derived row-by-row from the fact load.

6.8 Warehouse Implementation

The star schema was implemented in SQL Server within the `Warehouse` schema of the `Olist_DW` database. All foreign key constraints are enforced at the database level, and indexes were added on each foreign key column to support join performance during OLAP querying (Task 7). Table structures only were created at this stage; the ETL scripts that populate the dimensions and fact table from the validated `Staging` layer are documented in Task 5.

The successful creation of the warehouse schema was verified by querying the SQL Server system catalog. The validation query confirmed the existence of the six dimension tables and one central fact table required by the star schema design.

```
226 -- Confirm: should list 7 Warehouse tables
227 SELECT s.name AS schema_name, t.name AS table_name
228 FROM sys.tables t JOIN sys.schemas s ON s.schema_id = t.schema_id
229 WHERE s.name = N'Warehouse'
230 ORDER BY t.name;
231 GO
232
```

90 % No issues found

	schema_name	table_name
1	Warehouse	Dim_Customer
2	Warehouse	Dim_Date
3	Warehouse	Dim_Location
4	Warehouse	Dim_Payment
5	Warehouse	Dim_Product
6	Warehouse	Dim_Seller
7	Warehouse	Fact_Order_Sales

Figure 14: Validation of physically created Warehouse schema tables in SQL Server

The physical implementation follows the logical design by creating six dimension tables and one central fact table within the Warehouse schema. Figure 15 shows the physically created warehouse tables in SSMS Object Explorer, and Figure 16 shows an example of the table creation script.

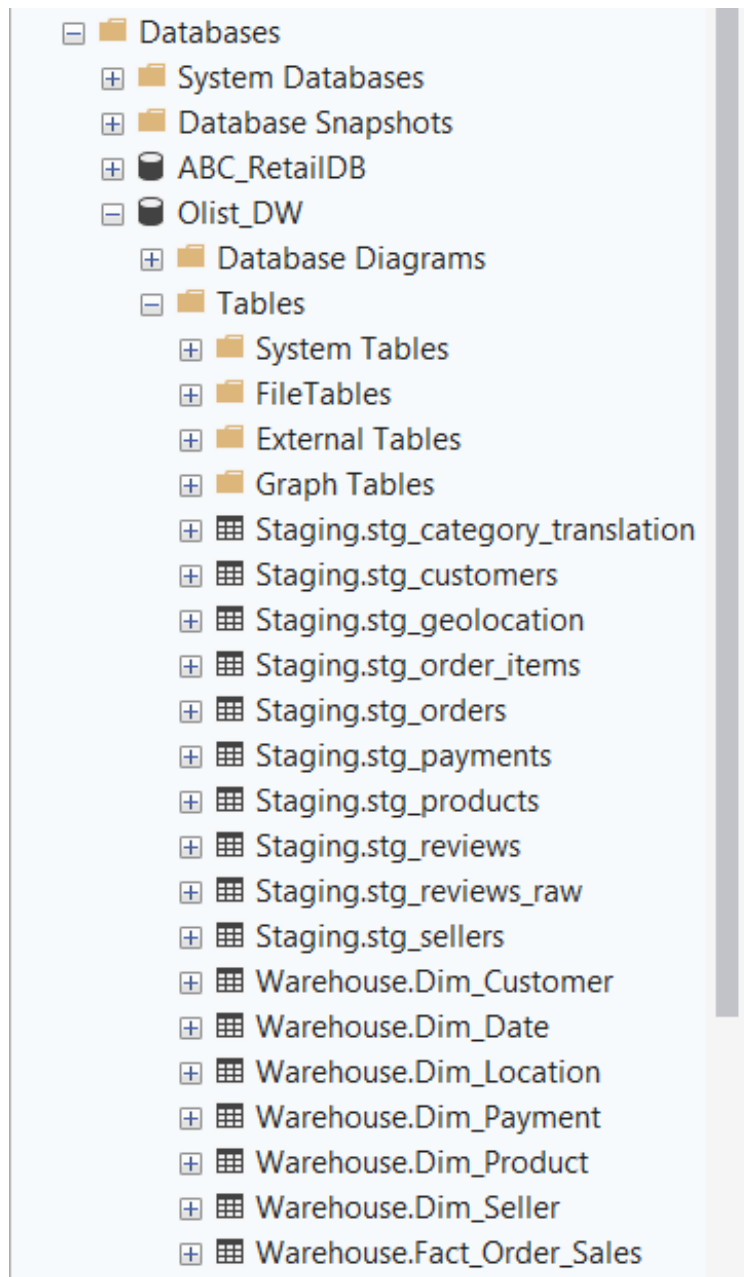


Figure 15: Physically created warehouse tables in SSMS Object Explorer

```

176  */
177  CREATE TABLE Warehouse.Fact_Order_Sales
178  (
179      order_sales_key          BIGINT IDENTITY(1,1) PRIMARY KEY,
180
181      -- Degenerate dimension: lets BI tools de-duplicate order-level measures
182      order_id                 NVARCHAR(50) NOT NULL,
183
184      -- Foreign keys
185      order_purchase_date_key  INT          NOT NULL,
186      customer_key             INT          NOT NULL,
187      customer_location_key    INT          NOT NULL,
188      product_key              INT          NOT NULL,
189      seller_key               INT          NOT NULL,
190      seller_location_key      INT          NOT NULL,
191      payment_key              INT          NOT NULL,
192
193      -- Line-grain measures (aggregated from unit-level order_items)
194      quantity                 INT          NOT NULL,
195      unit_price                DECIMAL(10,2) NOT NULL,
196      total_price               DECIMAL(12,2) NOT NULL,
197      unit_freight_value        DECIMAL(10,2) NOT NULL,
198      total_freight_value       DECIMAL(12,2) NOT NULL,
199
200      -- Order-level measures, replicated across the order's line items
201      -- (see the degenerate-dimension note above for correct aggregation)
202      order_total_payment_value DECIMAL(12,2) NULL,
203      order_total_installments INT          NULL,
204      delivery_duration_days    INT          NULL, -- NULL if not yet delivered
205      is_delivered_late         BIT          NULL, -- NULL if not yet delivered
206      review_score              TINYINT     NULL, -- NULL if no review submitted
207
208      CONSTRAINT FK_Fact_Date    FOREIGN KEY (order_purchase_date_key) REFERENCES Warehouse.Dim_Date(date_key),
209      CONSTRAINT FK_Fact_Customer FOREIGN KEY (customer_key)           REFERENCES Warehouse.Dim_Customer(customer_key),
210      CONSTRAINT FK_Fact_CustLoc  FOREIGN KEY (customer_location_key)  REFERENCES Warehouse.Dim_Location(location_key),
211      CONSTRAINT FK_Fact_Product  FOREIGN KEY (product_key)           REFERENCES Warehouse.Dim_Product(product_key),
212      CONSTRAINT FK_Fact_Seller   FOREIGN KEY (seller_key)            REFERENCES Warehouse.Dim_Seller(seller_key),
213      CONSTRAINT FK_Fact_SellLoc  FOREIGN KEY (seller_location_key)    REFERENCES Warehouse.Dim_Location(location_key),
214      CONSTRAINT FK_Fact_Payment  FOREIGN KEY (payment_key)           REFERENCES Warehouse.Dim_Payment(payment_key)
215  );
216  GO
217

```

Figure 16: Warehouse dimension table creation script (Dim_Customer)

The physical structure of the central fact table was further verified using the SQL Server `sp_help` system procedure. The output confirms the presence of the surrogate key, foreign keys linking to dimension tables, and indexing structures required for analytical querying.

SQLQuery2...SINDU (55)* X DESKTOP-8DB...schema_olist 05_create_wa...ASINDU (91)) SQLQuery1.s...ASINDU (65))*

EXEC sp_help 'Warehouse.Fact_Order_Sales';

82 % No issues found Ln: 1, Ch: 1 (42 chars, 1 lines) TABS CRLF UTF-8

Results Messages

Column_name	Type	Computed	Length	Prec	Scale	Nullable	TrimTrailingBlanks	FixedLenNullInSource	Collation
order_sales_key	bigint	no	8	19	0	no	(n/a)	(n/a)	NULL
order_id	nvarchar	no	100			no	(n/a)	(n/a)	Latin1_General_CI_AS
order_purchase_date_key	int	no	4	10	0	no	(n/a)	(n/a)	NULL
customer_key	int	no	4	10	0	no	(n/a)	(n/a)	NULL
customer_location_key	int	no	4	10	0	no	(n/a)	(n/a)	NULL
product_key	int	no	4	10	0	no	(n/a)	(n/a)	NULL
seller_key	int	no	4	10	0	no	(n/a)	(n/a)	NULL
seller_location_key	int	no	4	10	0	no	(n/a)	(n/a)	NULL
payment_key	int	no	4	10	0	no	(n/a)	(n/a)	NULL

Identity	Seed	Increment	Not For Replication
1	order_sales_key	1	0

RowGuidCol

1 No rowguidcol column defined.

Data_located_on_filegroup

1 PRIMARY

index_name	index_description	index_keys
IX_Fact_Customer	nonclustered located on PRIMARY	customer_key
IX_Fact_Date	nonclustered located on PRIMARY	order_purchase_date_key
IX_Fact_OrderId	nonclustered located on PRIMARY	order_id
IX_Fact_Product	nonclustered located on PRIMARY	product_key
IX_Fact_Seller	nonclustered located on PRIMARY	seller_key
PK_Fact_Order	clustered, unique, primary key lo...	order_sales_key

constraint_type	constraint_name	delete_action	update_action	status_enabled	status_for_replication	constraint_keys
FOREIGN KEY	FK_Fact_CustLoc	No Action	No Action	Enabled	Is_For_Replication	customer_location_key
FOREIGN KEY	FK_Fact_Custo...	No Action	No Action	Enabled	Is_For_Replication	customer_key
FOREIGN KEY	FK_Fact_Date	No Action	No Action	Enabled	Is_For_Replication	order_purchase_date...
FOREIGN KEY	FK_Fact_Paym...	No Action	No Action	Enabled	Is_For_Replication	payment_key
FOREIGN KEY	FK_Fact_Product	No Action	No Action	Enabled	Is_For_Replication	product_key

Figure 17: Physical structure and constraints of Fact_Order_Sales verified using SQL Server sp_help

7 Task 5: ETL Development

7.1 ETL Overview

The ETL process was implemented in Microsoft SQL Server using T-SQL, because the raw data was already ingested into the **Staging** schema of the **Olist.DW** database. Keeping extraction, transformation and loading inside the database engine allowed set-based transformations, direct enforcement of the foreign key constraints defined in Task 4, and straightforward reconciliation queries against the staging tables.

1. **Extract:** The nine Olist source files (customers, products, category translation, sellers, geolocation, orders, order items, payments, reviews) were loaded unchanged into **Staging.stg.*** tables.
2. **Transform:** Cleaning, standardisation, type conversion, deduplication, aggregation and derived attributes were applied while moving data from staging into the warehouse.
3. **Load:** The six dimension tables were loaded first, then the fact table, so that surrogate keys are available for the fact-table lookups.

7.2 Extraction Method and Source Connection

Sources. The nine Olist CSV files described in Task 2 are stored in a local folder on the machine hosting SQL Server (C:\olist_dataset\archive\).

Method. Extraction is performed by the T-SQL script **03.load_staging.sql**, which loads each file with **BULK INSERT** into its staging table in the **Staging** schema of **Olist.DW**. The SSMS Import Flat File wizard was tried first but was abandoned, because it failed with a data-conversion error on blank date values when typed columns were used. **BULK INSERT** combined with all-NVARCHAR staging tables avoids this.

Connection. The load runs inside the SQL Server instance, so no separate connection string is needed. Because **BULK INSERT** is executed by the SQL Server service account and not by the logged-in user, the CSV files must be in a plain folder that the service can read, not in a personal folder such as OneDrive or Desktop.

Process. The script keeps a list of the nine file-to-table pairs and processes them in a loop. For each file it truncates the target table, bulk loads the file, counts the rows and prints an **OK** or **FAIL** line, using

TRY...CATCH so that one failing file does not stop the rest. This is a full load, which suits a static historical dataset, and it makes the script safe to re-run. Table 18 lists the options used.

Option	Purpose
FORMAT = 'CSV', FIELDQUOTE = '"'	Applies CSV quoting rules. Without them, quote characters are stored in the data, keys differ between tables and joins lose rows.
FIELDTERMINATOR = ','	Column separator
ROWTERMINATOR = '0x0a'	Line-feed row ending used by the source files
CODEPAGE = '65001'	UTF-8, so Portuguese characters are preserved
FIRSTROW = 2	Skips the header row
TABLOCK	Faster bulk loading

Table 18: BULK INSERT options used for extraction

Extraction validation. The script `04_verify_staging.sql` then compares each staging table's row count with the expected source count (for example 99,441 orders and 112,650 order items), checks that no key values contain stray quote characters, and checks join integrity between the staging tables. The results are the staging validation shown in Task 2.

7.3 Staging Layer Design

All staging columns were deliberately created as `NVARCHAR`, including numeric and date fields. This preserves the source values exactly as delivered and prevents a whole load from failing because of a single malformed value. Data type enforcement and business-rule transformations are therefore performed in the ETL step, using `TRY_CAST`, which returns `NULL` for unconvertible values instead of raising an error, so bad values can be counted and handled explicitly rather than silently rejected.

7.4 Data Profiling and Cleaning Rules

Before loading, the staging tables were profiled with the script `05a_profile_staging.sql`. Table 19 lists the issues found and the rule applied to each in the ETL.

Source	Issue found	Rows	Rule applied
customers	Text with inconsistent case or surrounding spaces	0	Trim spaces; city to lower case; state to upper case
customers, sellers	Zip prefix not present in the location dimension	278, 7	Location lookup falls back to unknown member (key -1)
products	Missing category name	610	Category set to Unknown
products	Category with no English translation	13	English name falls back to the Portuguese name
products	Weight or dimensions missing or non-numeric	2, 2	Converted with TRY_CAST; left NULL
geolocation	Several coordinate pairs for the same zip prefix (zip prefixes affected)	17,781	Aggregated to one row per zip prefix (average coordinates, most frequent city and state)
geolocation	Non-numeric latitude or longitude	0	Excluded from coordinate averaging
payments	Payment type not_defined	3	Kept as its own payment-type member
orders	Not delivered, so no delivery date	2,965	Delivery measures left NULL (as designed in Task 4)
order items	Groups of records sharing order, product and seller	7,088	Aggregated to one fact row; quantity = COUNT(*)
reviews	Orders with more than one review	547	Reduced to one review per order in the fact load (Part B)

Table 19: Data profiling findings and cleaning rules

ord	check_name	n
1	customers: total rows	99441
2	customers: distinct customer_id	99441
3	customers: distinct customer_unique_id	96096
4	customers: blank/NULL city	0
5	customers: city with leading/trailing spaces or upper/lower case	0
6	customers: zip prefix not found in Dim_Location	278
7	products: total rows	32951
8	products: blank/NULL category	610
9	products: category with no English translation	13
10	products: weight missing or not numeric	2
11	products: any dimension (L/H/W) missing or not numeric	2
12	sellers: total rows	3095
13	sellers: distinct seller_id	3095
14	sellers: zip prefix not found in Dim_Location	7
15	geolocation: total rows	1000163
16	geolocation: distinct zip prefixes	19015
17	geolocation: zip prefixes with more than one distinct city and state	17781
18	geolocation: lat/long missing or not numeric	0
19	orders: total rows	99441
20	orders: purchase timestamp not a valid date	0
21	orders: delivered status but no delivery date	8
22	orders: not delivered (delivery measures will be NULL)	2965
23	order_items: total rows	112650
24	order_items: distinct (order, product, seller) groups	102425
25	order_items: groups with more than one row (aggregated)	7088
26	order_items: price not numeric	0
27	payments: total rows	103886
28	payments: distinct payment types	5
29	payments: type = not_defined	3
30	payments: orders with more than one payment record	2961
31	reviews: total rows	99224
32	reviews: orders with more than one review	547
33	reviews: score outside 1-5 or not numeric	0

Figure 18: Staging profiling results (output of 05a_profile_staging.sql)

7.5 ETL Workflow

Figure 20 shows the workflow from the staging layer to the warehouse star schema.

OLIST E-COMMERCE – ETL WORKFLOW

Figure 13 - Staging data to the warehouse star schema

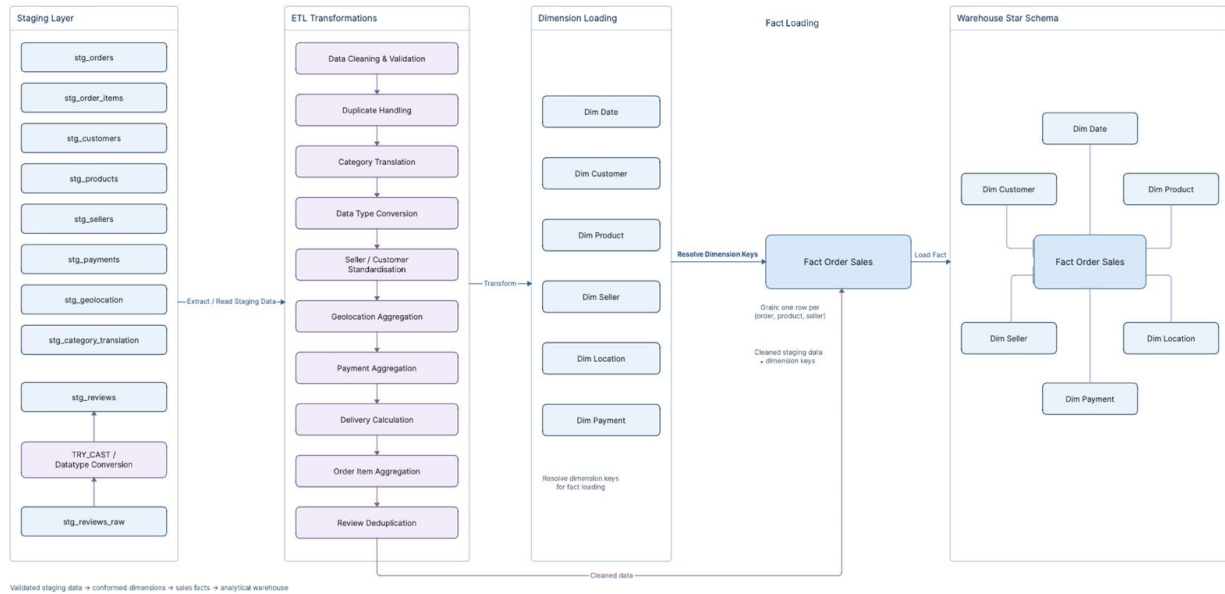


Figure 19: Operational entities of the Olist e-commerce system

```

06_load_dim...SINDU (83)  SQLQuery3.sq...ASINDU (56)  SQLQuery2.s...ASINDU (55))*  DESKTOP-8DB...schema_olist
141  /* ----- 6. Dim_Payment ----- */
142  INSERT INTO Warehouse.Dim_Payment (payment_type)
143  SELECT DISTINCT LOWER(LTRIM(RTRIM(payment_type)))
144  FROM Staging.stg_payments
145  WHERE payment_type IS NOT NULL AND LTRIM(RTRIM(payment_type)) <> '';
146  GO
147
148  /* ----- 7. Quick verification ----- */
149  SELECT 'Dim_Customer' AS tbl, COUNT(*) AS total_rows, SUM(CASE WHEN customer_key = -1 THEN 1 ELSE 0 END) AS unknown_rows F
150  UNION ALL SELECT 'Dim_Product', COUNT(*), SUM(CASE WHEN product_key = -1 THEN 1 ELSE 0 END) FROM Warehouse.Dim_Product
151  UNION ALL SELECT 'Dim_Seller', COUNT(*), SUM(CASE WHEN seller_key = -1 THEN 1 ELSE 0 END) FROM Warehouse.Dim_Seller
82 %  No issues found
Messages
Changed language setting to us_english.
(0 rows affected)
(0 rows affected)
(0 rows affected)
(0 rows affected)
(0 rows affected)
(5 rows affected)
(0 rows affected)
Checking identity information: current identity value 'NULL'.
DBCC execution completed. If DBCC printed error messages, contact your system administrator.
Checking identity information: current identity value 'NULL'.
DBCC execution completed. If DBCC printed error messages, contact your system administrator.
Checking identity information: current identity value 'NULL'.
DBCC execution completed. If DBCC printed error messages, contact your system administrator.
Checking identity information: current identity value 'NULL'.
DBCC execution completed. If DBCC printed error messages, contact your system administrator.
Checking identity information: current identity value 'NULL'.
DBCC execution completed. If DBCC printed error messages, contact your system administrator.
Checking identity information: current identity value '5'.
DBCC execution completed. If DBCC printed error messages, contact your system administrator.
(1096 rows affected)
(99441 rows affected)
(32951 rows affected)
(3095 rows affected)
(19015 rows affected)
(5 rows affected)
Completion time: 2026-09-29T13:27:30.3279445+05:30

```

Figure 20: ETL workflow from Staging layer to Warehouse star schema

7.6 Source-to-Target Mapping

Table 20 summarises how each dimension is populated. The dimension load is implemented in `06_load_dimensions.sql`.

Target	Source	Key transformations
Dim_Date	Generated (range from <code>stg_orders</code>)	One row per calendar day; integer key YYYYMMDD; day, month, quarter, year attributes; weekend flag
Dim_Customer	<code>stg_customers</code>	Trim; city lower case, state upper case; blank values replaced by unknown / NA ; duplicates removed
Dim_Product	<code>stg_products</code> , <code>stg_category_translation</code>	Left join to English category names (fallback: Portuguese name, then Unknown); text measurements converted to <code>DECIMAL(10,2)</code>
Dim_Seller	<code>stg_sellers</code>	Trim; city lower case, state upper case; blank values replaced by defaults
Dim_Location	<code>stg_geolocation</code>	One row per zip prefix: average latitude and longitude (<code>DECIMAL(10,6)</code>), most frequent city and state
Dim_Payment	<code>stg_payments</code>	Distinct, trimmed, lower-case payment types

Table 20: Staging-to-dimension mapping

7.7 Load Order and Re-runability

Dimensions are loaded before the fact table because fact rows need dimension surrogate keys. The script can be re-run safely: it first clears the fact table, then deletes every dimension row except the unknown member (key -1) and reseeds the identity column. `TRUNCATE TABLE` was deliberately not used, because SQL Server does not allow truncating a table referenced by a foreign key, and because truncation would also remove the unknown-member rows required by the design. Surrogate keys are generated by the `IDENTITY` property and are never inserted manually.

7.8 Dim_Date Generation

`Dim_Date` is generated independently of the transactional data, as specified in Task 4. The range starts on 1 January of the year of the earliest order purchase and ends on 31 December of the year of the latest estimated delivery date, so that every purchase, delivery and estimated delivery date falls inside the dimension. The date key uses the integer form `YYYYMMDD` (for example, 20180118 for 18 January 2018). The session language is set to `us_english` so that day and month names and the weekday numbering (Sunday = 1) are deterministic; Saturday and Sunday are flagged as weekend days.

7.9 Dim_Location Transformation

The geolocation source contains about one million rows for roughly nineteen thousand zip prefixes, with many different coordinate pairs for the same prefix. Loading it with `SELECT DISTINCT` would keep all of them, and every fact-table lookup by zip prefix would then match several dimension rows and multiply the sales measures. The ETL therefore collapses the source to exactly one row per zip prefix, averaging the coordinates and choosing the most frequently occurring city and state spelling:

```
;WITH g AS (
    SELECT LTRIM(RTRIM(geolocation_zip_code_prefix)) AS zip,
           LOWER(LTRIM(RTRIM(geolocation_city))) AS city,
           UPPER(LTRIM(RTRIM(geolocation_state))) AS state,
           TRY_CAST(geolocation_lat AS DECIMAL(10,6)) AS lat,
           TRY_CAST(geolocation_lng AS DECIMAL(10,6)) AS lng
    FROM Staging.stg_geolocation
),
coords AS (
    SELECT zip, AVG(lat) AS latitude, AVG(lng) AS longitude
    FROM g WHERE lat IS NOT NULL AND lng IS NOT NULL
    GROUP BY zip
),
city_pick AS (
    SELECT zip, city, state,
           ROW_NUMBER() OVER (PARTITION BY zip
                               ORDER BY COUNT(*) DESC, city) AS rn
    FROM g GROUP BY zip, city, state
)
INSERT INTO Warehouse.Dim_Location (...)
SELECT ... FROM city_pick cp JOIN coords co ON co.zip = cp.zip
WHERE cp.rn = 1;
```

Because `Dim_Location` is the role-playing dimension used for both customers and sellers, this one-row-per-zip rule is essential to keeping the fact grain intact.

7.10 Dimension Load Validation

After loading, the row count and unknown-member presence of each dimension were checked. Table 21 compares the loaded rows with the staging source counts and confirms the presence of unknown-member

records.

Dimension	Staging source count	Rows loaded (excl. -1)	Unknown row
Dim_Date	n/a (generated)	1,096	Yes
Dim_Customer	99,441	99,441	Yes
Dim_Product	32,951	32,951	Yes
Dim_Seller	3,095	3,095	Yes
Dim_Location	19,015	19,015	Yes
Dim_Payment	5	5	Yes

Table 21: Dimension row-count validation

```

146  --
147  --
148  /* ----- 7. Quick verification ----- */
149  SELECT 'Dim_Customer' AS tbl, COUNT(*) AS total_rows, SUM(CASE WHEN customer_key = -1 THEN 1 ELSE 0 END) AS unknown_rows FROM Warehouse.Dim_Customer
150  UNION ALL SELECT 'Dim_Product', COUNT(*), SUM(CASE WHEN product_key = -1 THEN 1 ELSE 0 END) FROM Warehouse.Dim_Product
151  UNION ALL SELECT 'Dim_Seller', COUNT(*), SUM(CASE WHEN seller_key = -1 THEN 1 ELSE 0 END) FROM Warehouse.Dim_Seller
152  UNION ALL SELECT 'Dim_Location', COUNT(*), SUM(CASE WHEN location_key = -1 THEN 1 ELSE 0 END) FROM Warehouse.Dim_Location
153  UNION ALL SELECT 'Dim_Payment', COUNT(*), SUM(CASE WHEN payment_key = -1 THEN 1 ELSE 0 END) FROM Warehouse.Dim_Payment
154  UNION ALL SELECT 'Dim_Date', COUNT(*), SUM(CASE WHEN date_key = -1 THEN 1 ELSE 0 END) FROM Warehouse.Dim_Date;
155

```

tbl	total_rows	unknown_rows
1 Dim_Customer	99442	1
2 Dim_Product	32952	1
3 Dim_Seller	3096	1
4 Dim_Location	19016	1
5 Dim_Payment	6	1
6 Dim_Date	1097	1

Figure 21: Dimension row counts and unknown-member check (SSMS output)

A further check confirmed the one-row-per-zip-prefix rule for Dim.Location; the query below returned zero rows.

```

SELECT zip_code_prefix, COUNT(*) AS n
FROM Warehouse.Dim_Location
GROUP BY zip_code_prefix
HAVING COUNT(*) > 1;

```

```

155  --
156  -- Must return 0 rows (one location row per zip prefix)
157  SELECT zip_code_prefix, COUNT(*) AS n
158  FROM Warehouse.Dim_Location
159  GROUP BY zip_code_prefix
160  HAVING COUNT(*) > 1;
161  GO
162

```

zip_code_prefix	n
-----------------	---

Figure 22: Validation showing no duplicate zip prefixes in Dim.Location

7.11 Fact Table Load

The fact table `Fact_Order_Sales` was loaded by the script `07_load_fact.sql` in a single set-based `INSERT ... SELECT`, after all dimensions had been populated. The script is re-runnable: it first clears the fact table and reseeds its identity column. The load is built from a chain of common table expressions, each preparing one part of the fact row:

- **items:** aggregates the source order-item records to the fact grain (order, product, seller).
- **ord:** converts the order dates from text to `DATETIME`.
- **pay_total** and **pay_primary:** derive the order-level payment totals and the primary payment type.
- **rev:** reduces the reviews to one per order.

Table 22 summarises how each fact column is derived.

Fact column	Derivation
<code>order_id</code>	Trimmed source order identifier (degenerate dimension)
<code>quantity</code>	<code>COUNT(*)</code> of source order-item records in the (order, product, seller) group
<code>unit_price</code> , <code>unit_freight_value</code>	<code>MAX</code> of the converted value; safe because both are constant within a group (verified in Task 4)
<code>total_price</code> , <code>total_freight_value</code>	<code>quantity</code> $\times$ the unit value
<code>order_total_payment_value</code> <code>order_total_installments</code>	<code>SUM</code> over all payment records of the order; 0 when the order has no payment record
<code>payment_key</code>	Key of the payment record with the highest <code>payment_value</code> (ties resolved by the lowest sequence number); <code>-1</code> if none
<code>review_score</code>	Score of the most recent review of the order (by creation date), kept only if between 1 and 5; <code>NULL</code> if the order has no review
<code>delivery_duration_days</code>	Whole calendar days from purchase date to delivery date; <code>NULL</code> if not delivered
<code>is_delivered_late</code>	1 if the delivery date is after the estimated delivery date, otherwise 0; <code>NULL</code> if not delivered
Dimension keys	<code>LEFT JOIN</code> on the business key; <code>ISNULL(..., -1)</code> for unmatched values. Customer and seller locations are resolved through the zip prefix stored in the customer and seller dimensions

Table 22: Derivation of the fact table columns

Aggregation to the fact grain

The aggregation promised in the Task 4 grain definition is implemented as follows:

```
SELECT LTRIM(RTRIM(order_id)) AS order_id,  
       LTRIM(RTRIM(product_id)) AS product_id,  
       LTRIM(RTRIM(seller_id)) AS seller_id,  
       COUNT(*) AS quantity,  
       MAX(TRY_CAST(price AS DECIMAL(12,2))) AS unit_price,  
       MAX(TRY_CAST(freight_value AS DECIMAL(12,2))) AS unit_freight_value  
FROM Staging.stg_order_items  
GROUP BY LTRIM(RTRIM(order_id)), LTRIM(RTRIM(product_id)),  
         LTRIM(RTRIM(seller_id))
```

Primary payment type

For orders with several payment records, the primary payment type is the record with the highest payment value:

```

ROW_NUMBER() OVER (
    PARTITION BY LTRIM(RTRIM(order_id))
    ORDER BY TRY_CAST(payment_value AS DECIMAL(12,2)) DESC,
             TRY_CAST(payment_sequential AS INT) ASC) AS rn

```

Only the row with `rn = 1` is used for `payment_key`, while `order_total_payment_value` and `order_total_installments` are summed over all of the order's payment records, as defined in Task 4.

One review per order

Some orders have more than one review (547 in the profiling results). Joining the review table directly would duplicate the fact rows of those orders. The ETL therefore keeps a single review per order, the most recent by creation date, so the fact grain is preserved.

Date handling

All date conversions wrap the source text in `NULLIF(..., '')` before `TRY_CAST`. In SQL Server an empty string converts to 1900-01-01 rather than `NULL`, which would otherwise have produced false dates and incorrect delivery durations.

7.12 Fact Load Results

The load completed without errors and inserted 102,425 rows, matching the number of distinct (order, product, seller) groups identified during profiling.

```

19
20  /* ----- 1. Load ----- */
21  ;WITH items AS      -- aggregate source order-item records to the fact grain
22  (
23      SELECT
24          LTRIM(RTRIM(order_id)) AS order_id,
25          LTRIM(RTRIM(product_id)) AS product_id,
26          LTRIM(RTRIM(seller_id)) AS seller_id,
27          COUNT(*) AS quantity,
28          MAX(TRY_CAST(price AS DECIMAL(12,2))) AS unit_price,
29          MAX(TRY_CAST(freight_value AS DECIMAL(12,2))) AS unit_freight_value
30      FROM Staging.stg_order_items
31      GROUP BY LTRIM(RTRIM(order_id)), LTRIM(RTRIM(product_id)), LTRIM(RTRIM(seller_id))
32  ),
33  ord AS              -- typed order dates (NULLIF guards against '' -> 1900-01-01)

```

120 % ● No issues found

Messages

Changed language setting to us_english.

(0 rows affected)

Checking identity information: current identity value 'NULL'.

DBCC execution completed. If DBCC printed error messages, contact your system administrator.

(102425 rows affected)

Completion time: 2026-09-29T14:52:16.8363678+05:30

Figure 23: Fact load script (aggregation step) and successful execution: 102,425 rows inserted

Table 23 summarises the loaded fact table, and Figure 24 shows the corresponding SSMS output.

Measure	Value
Fact rows loaded	102,425
Total units (SUM(quantity))	112,650
Total revenue (SUM(total_price))	13,591,643.70
Total freight (SUM(total_freight_value))	2,251,909.54
Rows using unknown customer location	283
Rows using unknown seller location	225
Rows using unknown payment	1
Rows without a review score	797
Rows without delivery measures (not delivered)	2,230

Table 23: Fact table load summary

```

140
141 /* ----- 2. Quick check (full validation is in 08_validate_etl.sql) ----- */
142 SELECT
143     COUNT(*) AS fact_rows, -- expect 102,425
144     SUM(quantity) AS total_units, -- expect = staging order_items rows
145     SUM(total_price) AS fact_revenue, -- expect = SUM(price) in staging
146     SUM(total_freight_value) AS fact_freight, -- expect = SUM(freight_value) in staging
147     SUM(CASE WHEN customer_location_key = -1 THEN 1 ELSE 0 END) AS unknown_customer_location,
148     SUM(CASE WHEN seller_location_key = -1 THEN 1 ELSE 0 END) AS unknown_seller_location,
149     SUM(CASE WHEN payment_key = -1 THEN 1 ELSE 0 END) AS unknown_payment,
150     SUM(CASE WHEN review_score IS NULL THEN 1 ELSE 0 END) AS no_review,
151     SUM(CASE WHEN delivery_duration_days IS NULL THEN 1 ELSE 0 END) AS not_delivered
152 FROM Warehouse.Fact_Order_Sales;
153
154 SELECT

```

	fact_rows	total_units	fact_revenue	fact_freight	unknown_customer_location	unknown_seller_location	unknown_payment	no_review	not_delivered
1	102425	112650	13591643.70	2251909.54	283	225	1	797	2230

Figure 24: Fact table load summary query (SSMS output)

The unknown-member counts are consistent with the profiling results. The 283 fact rows with an unknown customer location belong to customers whose zip prefix does not exist in `Dim.Location` (278 customers), and the 225 rows with an unknown seller location belong to the 7 sellers with a missing zip prefix. The single row with an unknown payment belongs to an order that has no payment record in staging (see check 18 in Table 25). Rows without a review score or delivery measures reflect orders that have no review or are not yet delivered; these measures are nullable by design. The 2,230 undelivered fact rows are fewer than the 2,965 undelivered orders found in profiling because the fact table only contains orders that have at least one order-item record.

7.13 Reconciliation with the Staging Layer

To confirm that no data was lost or duplicated by the aggregation and joins, the warehouse totals were compared with the staging totals. Table 24 shows that units, revenue and freight all match exactly.

Measure	Staging	Warehouse	Result
Order-item records / total units	112,650	112,650	Match
Revenue (price)	13,591,643.70	13,591,643.70	Match
Freight value	2,251,909.54	2,251,909.54	Match

Table 24: Reconciliation of staging and warehouse totals

```

153
154 SELECT
155     COUNT(*) AS staging_item_rows,
156     SUM(TRY_CAST(price AS DECIMAL(12,2))) AS staging_revenue,
157     SUM(TRY_CAST(freight_value AS DECIMAL(12,2))) AS staging_freight
158 FROM Staging.stg_order_items;
159 GO
160

```

120 % No issues found

Results Messages

	staging_item_rows	staging_revenue	staging_freight
1	112650	13591643.70	2251909.54

Figure 25: Staging totals used for the reconciliation (SSMS output)

Matching revenue and freight confirm that the join to the payment, review and dimension tables did not multiply any fact rows, since a single duplicated line would have inflated the totals.

7.14 Integrity Validation

The script `08_validate_etl.sql` runs the remaining checks and reports a PASS or FAIL for each. The results are shown in Table 25 and Figure 26.

#	Check	Expected	Actual	Status
1	Fact rows equal distinct (order, product, seller) groups in staging	102,425	102,425	PASS
2	Total units equal staging order-item records	112,650	112,650	PASS
3	Revenue equals staging revenue	13,591,643.70	13,591,643.70	PASS
4	Freight equals staging freight	2,251,909.54	2,251,909.54	PASS
5	Distinct orders equal staging order-item orders	98,666	98,666	PASS
6	Duplicate grain rows	0	0	PASS
7-13	Orphan foreign keys (seven checks, one per key)	0	0	PASS
14	Orders with inconsistent review score across lines	0	0	PASS
15	Orders with inconsistent payment total across lines	0	0	PASS
16	Payment total (one value per order) equals staging payments	15,846,280.17	15,846,280.17	PASS
17	Review score outside 1-5	0	0	PASS
18	Unknown payment rows equal orders with no payment record	1	1	PASS

Table 25: ETL integrity validation results

90 %		No issues found			
Results		Messages			
ord	check_name	expected	actual	status	
1	1	Fact rows = distinct (order, product, seller) groups i...	102425.00	102425.00	PASS
2	2	Total units = staging order_items rows	112650.00	112650.00	PASS
3	3	Revenue: SUM(total_price) = staging SUM(price)	13591643.70	13591643.70	PASS
4	4	Freight: SUM(total_freight_value) = staging SUM(f...	2251909.54	2251909.54	PASS
5	5	Distinct orders in fact = distinct orders in staging or...	98666.00	98666.00	PASS
6	6	Duplicate grain rows (order_id, product_key, seller...	0.00	0.00	PASS
7	7	Orphan keys: order_purchase_date_key	0.00	0.00	PASS
8	8	Orphan keys: customer_key	0.00	0.00	PASS
9	9	Orphan keys: customer_location_key	0.00	0.00	PASS
10	10	Orphan keys: product_key	0.00	0.00	PASS
11	11	Orphan keys: seller_key	0.00	0.00	PASS
12	12	Orphan keys: seller_location_key	0.00	0.00	PASS
13	13	Orphan keys: payment_key	0.00	0.00	PASS
14	14	Orders with inconsistent review_score across their ...	0.00	0.00	PASS
15	15	Orders with inconsistent order_total_payment_val...	0.00	0.00	PASS
16	16	Payment total (one value per order) = staging SU...	15846280.17	15846280.17	PASS
17	17	review_score outside 1-5	0.00	0.00	PASS
18	18	Rows with payment_key = -1 = fact rows whose or...	1.00	1.00	PASS

fk	rows_using_unknown_member
1	order_purchase_date_key
2	customer_key
3	customer_location_key
4	product_key
5	seller_key
6	seller_location_key
7	payment_key

Figure 26: Output of 08.validate_etl.sql

Checks 6 and 7–13 verify that the declared grain holds and that every foreign key resolves to a dimension row, including the unknown member. Checks 14 and 15 confirm that the order-level measures replicated across the lines of an order are consistent, which is what allows a BI tool to de-duplicate them by **order_id** as described in Task 4. Check 16 confirms that when one value per order is taken, the payment totals in the warehouse equal the totals in staging.

7.15 Summary

The ETL process transformed the raw staging data into the star schema designed in Task 4. Profiling identified the data quality issues and the cleaning rules applied to them. The six dimensions were loaded first with standardised attributes, surrogate keys and unknown-member rows, and **Dim_Location** was reduced to one row per zip prefix to protect the fact grain. The fact table was then loaded at the (order, product, seller) grain with **quantity** derived by aggregation, order-level payment and review information attached without duplication, and delivery performance derived from the order dates. Reconciliation against staging showed identical unit, revenue and freight totals, and the integrity checks confirmed the grain and referential integrity of the warehouse, which is therefore ready for the data mart and BI dashboard in the following tasks.

8 Task 6: Data Mart Development

8.1 Purpose of the Data Mart

A data mart is a subject-oriented subset of the enterprise data warehouse designed for the needs of one department. The **Sales Performance Data Mart** was built for the sales and marketing team. It exposes only the sales-related facts and descriptive attributes that team needs, in a simple and analysis-ready form, instead of requiring analysts to join the fact table to six dimension tables and to understand the grain and replication rules described in Task 4.

8.2 Intended Users and Business Questions

Table 26 lists the users of the mart and the questions it is designed to answer.

User	Business question	Mart object
Sales manager	How do orders, revenue and average order value change month by month?	<code>vw_Sales_Detail</code>
Category manager	Which product categories generate the most revenue and units?	<code>Sales_Monthly_Category</code>
Regional marketing analyst	Which customer and seller states drive sales?	<code>vw_Sales_Detail</code>
Marketing analyst	Are sales stronger on weekends, or in particular quarters?	<code>vw_Sales_Detail</code>

Table 26: Users and business questions of the Sales Performance Data Mart

8.3 Design

The mart is implemented in its own `Mart` schema inside the `Olist.DW` database and reads only from the `Warehouse` star schema, so it always agrees with the enterprise warehouse. It contains two objects (Table 27).

Object	Type	Grain	Purpose
<code>Mart.vw_Sales_Detail</code>	View	One row per order, product and seller	Flattened, business-friendly detail with date, product, seller, customer and payment attributes joined in
<code>Mart.Sales_Monthly_Category</code>	Table	One row per year-month and product category	Pre-aggregated units, revenue and freight for fast category and trend reporting

Table 27: Objects of the Sales Performance Data Mart

The view removes the need for joins and gives the attributes readable names (for example `product_category` and `item_revenue`). The summary table stores a pre-aggregated copy of the data, which reduces the volume scanned by dashboards and reports.

Measures included and excluded

Table 28 defines the mart's metrics.

Metric	Definition
Units	<code>SUM(quantity)</code>
Item revenue	<code>SUM(total_price)</code> ; product sales value excluding freight
Freight	<code>SUM(total_freight_value)</code>
Orders	<code>COUNT(DISTINCT order_id)</code>
Average order value	Item revenue divided by distinct orders

Table 28: Metric definitions

Two design decisions follow from the fact table design in Task 4. First, the order-level measures that are replicated across the lines of an order (payment totals, review score and delivery measures) are deliberately **excluded** from this mart. Summing them across lines would overstate them, and they belong to other subject areas such as finance, customer experience and logistics, which could each have their own mart. Second, the number of orders is counted with `DISTINCT order_id`, and the `orders` column of the summary table is **not additive** across categories, because an order containing products from several categories is counted once in each of them. Totals of orders must therefore be taken from the view.

8.4 Implementation

The mart is created by `09_create_sales_mart.sql`, which is re-runnable. The view joins the fact table to the date, product, seller, customer and payment dimensions:

```
CREATE VIEW Mart.vw_Sales_Detail AS
SELECT
    f.order_sales_key, f.order_id,
    d.full_date        AS order_date,
    d.year             AS order_year,
    d.quarter          AS order_quarter,
    d.month_number     AS order_month_number,
    d.month_name       AS order_month_name,
    d.day_name         AS order_day_name,
    d.is_weekend       AS is_weekend_order,
    p.product_id,
    p.category_name_en AS product_category,
    s.seller_id, s.seller_city, s.seller_state,
    c.customer_unique_id, c.customer_city, c.customer_state,
    pay.payment_type   AS primary_payment_type,
    f.quantity         AS units,
    f.unit_price,
    f.total_price      AS item_revenue,
    f.total_freight_value AS freight_value,
    f.total_price + f.total_freight_value AS gross_value
FROM Warehouse.Fact_Order_Sales f
JOIN Warehouse.Dim_Date      d  ON d.date_key      = f.order_purchase_date_key
JOIN Warehouse.Dim_Product  p  ON p.product_key   = f.product_key
JOIN Warehouse.Dim_Seller   s  ON s.seller_key     = f.seller_key
JOIN Warehouse.Dim_Customer c  ON c.customer_key  = f.customer_key
JOIN Warehouse.Dim_Payment  pay ON pay.payment_key = f.payment_key;
```

The summary table is loaded from the fact table by grouping on year, month and product category; the full loading query is in `09_create_sales_mart.sql`.

Figure 27 shows the created objects in SSMS.

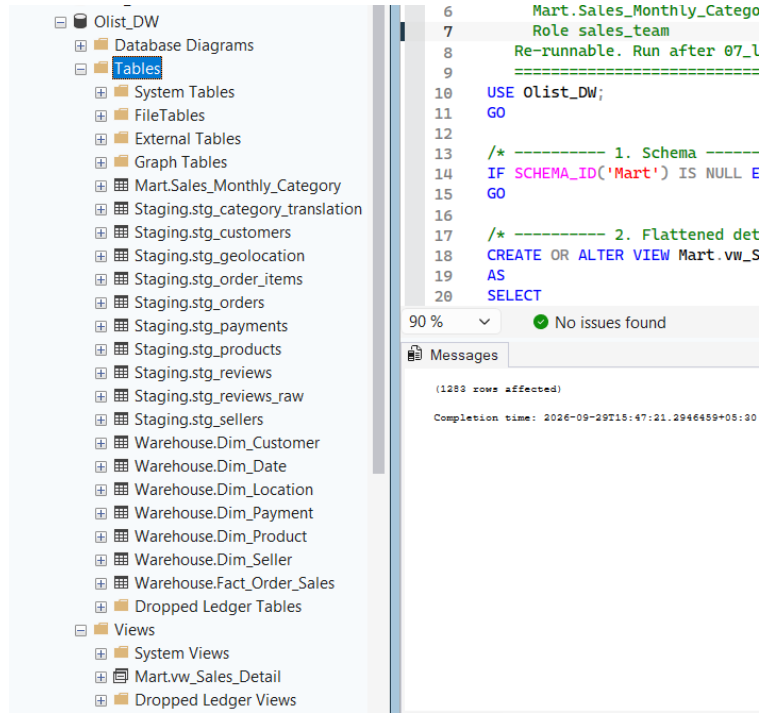


Figure 27: Sales Performance Data Mart objects in SSMS Object Explorer

8.5 Access Control

A database role `sales_team` was created and granted `SELECT` on the `Mart` schema only. Members of the sales and marketing team can therefore query the mart without any access to the underlying `Staging` or `Warehouse` schemas, which supports the principle of least privilege.

8.6 Validation

The mart was reconciled with the warehouse fact table using `10_mart_queries_validation.sql`. Table 29 compares the totals.

Metric	Warehouse	Mart view	Mart table	Status
Rows (fact / view)	102,425	102,425	n/a	PASS
Units	112,650	112,650	112,650	PASS
Item revenue	13,591,643.70	13,591,643.70	13,591,643.70	PASS
Freight	2,251,909.54	2,251,909.54	2,251,909.54	PASS
Distinct orders	98,666	98,666	n/a	PASS

Table 29: Reconciliation of the data mart with the warehouse

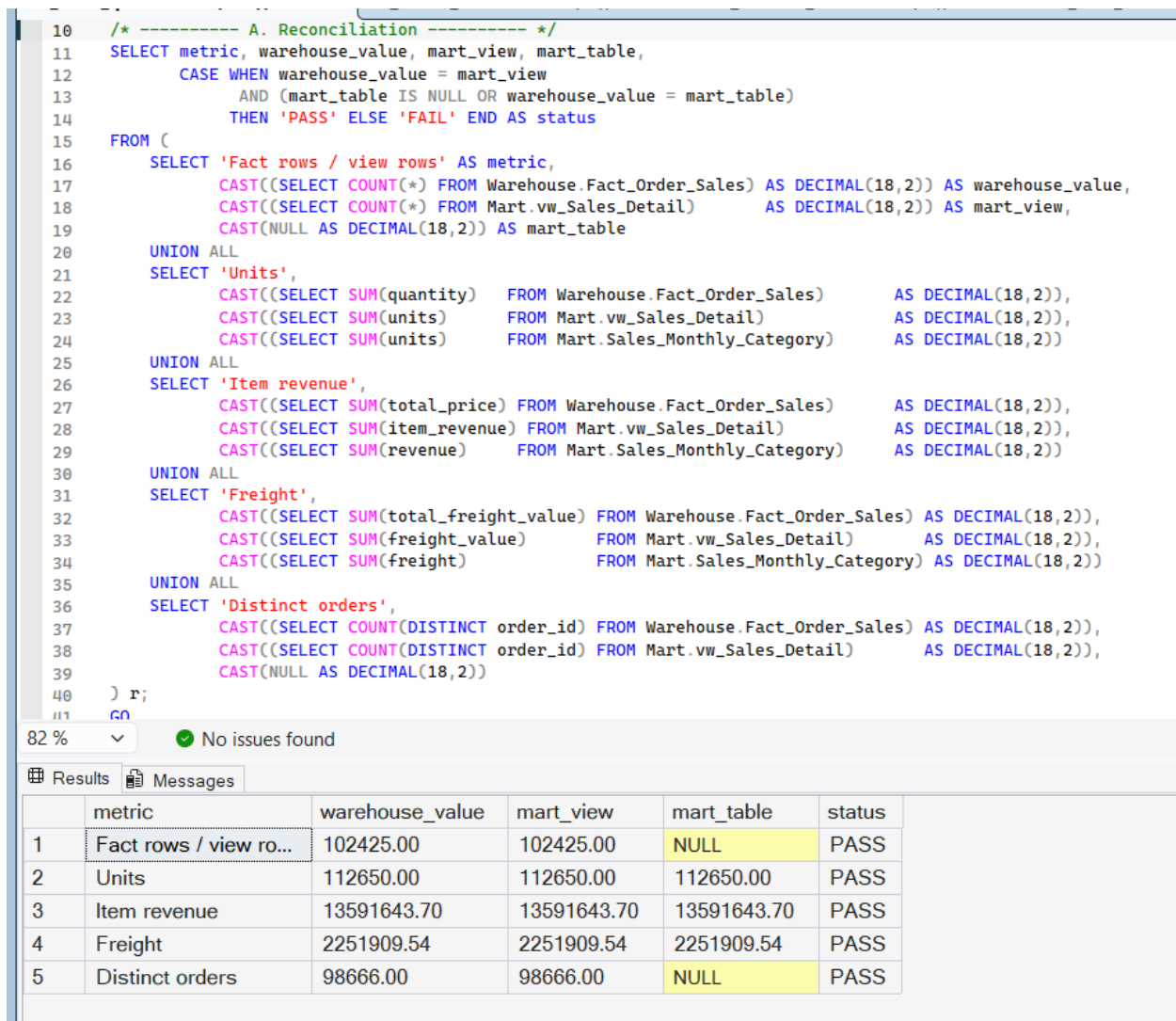


Figure 28: Data mart reconciliation results (SSMS output)

The summary table contains 1,283 rows, compared with 102,425 rows in the fact table (about 1.3 percent of its volume), and the view returns exactly the fact table's rows, confirming that the joins neither lose nor duplicate any sales lines.

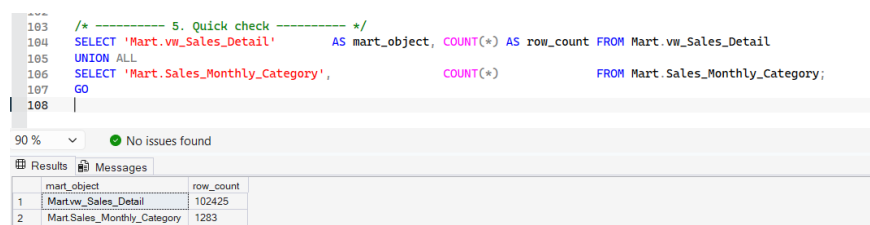
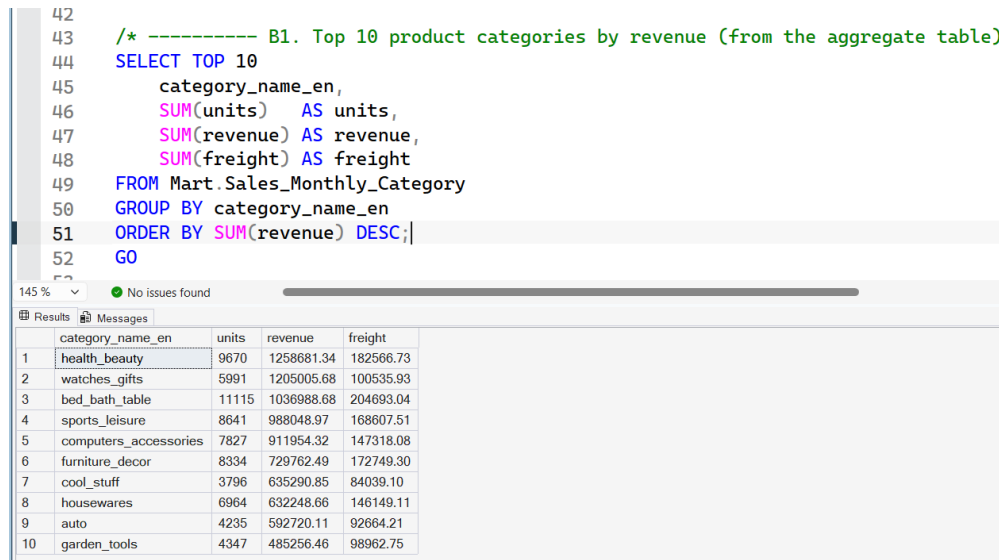


Figure 29: Row counts of the data mart objects after loading

8.7 Example Analyses

To demonstrate the mart, three queries typical of the sales team were run against it: the top ten product categories by revenue (from the summary table), the monthly trend of orders, revenue and average order value (from the view), and the top ten customer states by revenue (from the view). Their results are shown in Figures 30–32, and the interpretation of these results is developed in the business insights section (Task 8).



```
42
43  /* ----- B1. Top 10 product categories by revenue (from the aggregate table)
44  SELECT TOP 10
45      category_name_en,
46      SUM(units) AS units,
47      SUM(revenue) AS revenue,
48      SUM(freight) AS freight
49  FROM Mart.Sales_Monthly_Category
50  GROUP BY category_name_en
51  ORDER BY SUM(revenue) DESC;
52  GO
```

145 % No issues found

	category_name_en	units	revenue	freight
1	health_beauty	9670	1258681.34	182566.73
2	watches_gifts	5991	1205005.68	100535.93
3	bed_bath_table	11115	1036988.68	204693.04
4	sports_leisure	8641	988048.97	168607.51
5	computers_accessories	7827	911954.32	147318.08
6	furniture_decor	8334	729762.49	172749.30
7	cool_stuff	3796	635290.85	84039.10
8	housewares	6964	632248.66	146149.11
9	auto	4235	592720.11	92664.21
10	garden_tools	4347	485256.46	98962.75

Figure 30: Top 10 product categories by revenue

The summary table shows that **health_beauty** is the top category by revenue (1,258,681.34), followed by **watches_gifts** (1,205,005.68) and **bed_bath_table** (1,036,988.68). Among the ten categories, **bed_bath_table** sells the most units (11,115), so ranking by units and ranking by revenue give different orders.

74 % ✓ No issues found

Results Messages

	order_year	order_month_number	orders	units	revenue	avg_order_value
1	2016	9	3	6	267.36	89.12
2	2016	10	308	363	49507.66	160.74
3	2016	12	1	1	10.90	10.90
4	2017	1	789	955	120312.87	152.49
5	2017	2	1733	1951	247303.02	142.70
6	2017	3	2641	3000	374344.30	141.74
7	2017	4	2391	2684	359927.23	150.53
8	2017	5	3660	4136	506071.14	138.27
9	2017	6	3217	3583	433038.60	134.61
10	2017	7	3969	4519	498031.48	125.48
11	2017	8	4293	4910	573971.68	133.70
12	2017	9	4243	4831	624401.69	147.16
13	2017	10	4568	5322	664219.43	145.41
14	2017	11	7451	8665	1010271...	135.59
15	2017	12	5624	6308	743914.17	132.27
16	2018	1	7220	8208	950030.36	131.58
17	2018	2	6694	7672	844178.71	126.11
18	2018	3	7188	8217	983213.44	136.79
19	2018	4	6934	7975	996647.75	143.73
20	2018	5	6853	7925	996517.68	145.41
21	2018	6	6160	7078	865124.31	140.44
22	2018	7	6273	7092	895507.22	142.76
23	2018	8	6452	7248	854686.33	132.47
24	2018	9	1	1	145.00	145.00

Figure 31: Monthly orders, revenue and average order value

Monthly orders grow from 789 in January 2017 to a peak of 7,451 in November 2017, and then stay between roughly 6,200 and 7,200 per month from January to August 2018. The first and last months of the data (September to December 2016 and September 2018) contain very few orders and no orders were recorded in November 2016, so they are partial periods that must be treated with care in trend analysis.

```

68 SELECT TOP 10
69     customer_state,
70     COUNT(DISTINCT order_id) AS orders,
71     COUNT(DISTINCT customer_unique_id) AS customers,
72     SUM(item_revenue) AS revenue,
73     CAST(SUM(item_revenue) / COUNT(DISTINCT order_id) AS DECIMAL(10,2)) AS avg_order_value
74 FROM Mart.vw_Sales_Detail
75 GROUP BY customer_state
76 ORDER BY SUM(item_revenue) DESC;
77 GO
78

```

90 % ✔ No issues found

	customer_state	orders	customers	revenue	avg_order_value
1	SP	41375	39981	5202955.05	125.75
2	RJ	12762	12303	1824092.67	142.93
3	MG	11544	11178	1585308.03	137.33
4	RS	5432	5249	750304.02	138.13
5	PR	4998	4840	683083.76	136.67
6	SC	3612	3513	520553.34	144.12
7	BA	3358	3257	511349.99	152.28
8	DF	2125	2062	302603.94	142.40
9	GO	2007	1942	294591.95	146.78
10	ES	2025	1956	275037.31	135.82

Figure 32: Top 10 customer states by revenue

Sales are concentrated in a few states: São Paulo (SP) alone accounts for 5,202,955.05 of the total item revenue of 13,591,643.70 (about 38 percent), the top three states (SP, RJ and MG) for about 63 percent, and the ten states shown for about 88 percent. The number of customers is close to the number of orders in every state (for example 39,981 customers for 41,375 orders in SP), indicating that most customers place a single order.

8.8 Benefits

- **Simplicity:** analysts query one view with readable column names instead of joining seven tables.
- **Consistency:** metric definitions (revenue, units, orders, average order value) are fixed in one place, and the exclusion of replicated order-level measures prevents double counting.
- **Performance:** the pre-aggregated table answers category and monthly trend questions from far fewer rows than the fact table.
- **Security:** the sales team receives access to the mart schema only.
- **Reuse:** the mart is the data source for the dashboard in Task 7 and the insights in Task 8.

8.9 Limitations and Extensions

The summary table is a snapshot and must be reloaded after each warehouse refresh; in a production environment this would be scheduled after the ETL run. Other departments could be served by additional marts built in the same way, for example a logistics mart for delivery performance and a customer experience mart for review scores, both of which use the order-level measures that were deliberately left out of the sales mart.

9 Task 7: OLAP Analysis and Business Intelligence Dashboard Development

9.1 Overview

An interactive three-page dashboard was developed in Microsoft Power BI Desktop (*Olist_Sales_Dashboard.pbix*). The report is built on the `Mart.vw_Sales_Detail` data mart view (Task 6) joined to `Warehouse.Dim_Date`,

and follows the structure required by the assignment: Executive Summary, Trend Analysis and Interactive Analysis.

9.2 Data Model

Two tables were loaded into Power BI and renamed to keep measure formulas short: `Mart.vw_Sales_Detail` became `Sales`, and `Warehouse.Dim_Date` became `Dates`. An active many-to-one relationship links `Sales[order_date]` to `Dates[full_date]`, so every time-based visual can use the calendar attributes of the date dimension (year, quarter, month).

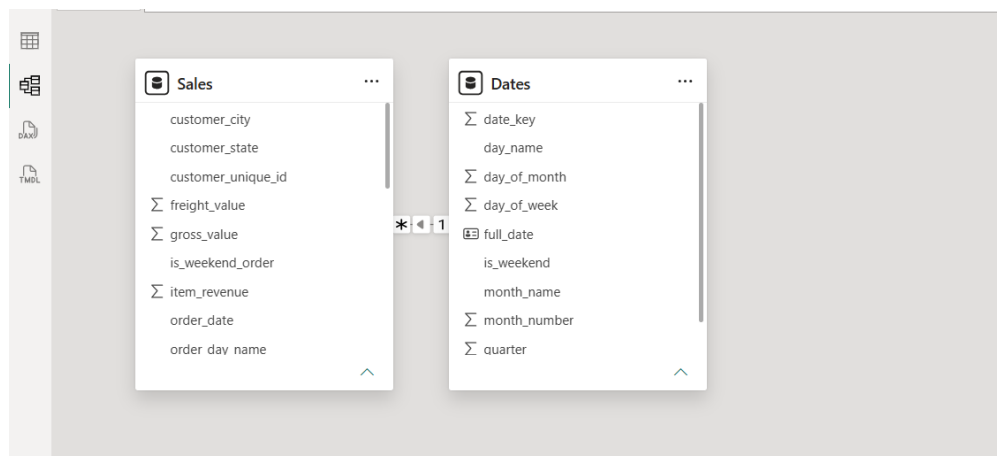


Figure 33: Power BI data model showing the relationship between Sales and Dates

9.3 DAX Measures

Five measures were created on the `Sales` table. Their values were checked against the totals returned by the source SQL view before being used in any visual.

Measure	DAX definition	Value
Total Revenue	<code>SUM(Sales[item_revenue])</code>	13,591,643.70
Total Orders	<code>DISTINCTCOUNT(Sales[order_id])</code>	98,666
Total Units	<code>SUM(Sales[units])</code>	112,650
Avg Order Value	<code>DIVIDE([Total Revenue], [Total Orders])</code>	137.75
Avg Units per Order	<code>DIVIDE([Total Units], [Total Orders])</code>	1.14

Table 30: DAX measures used in the dashboard

`Total Orders` uses `DISTINCTCOUNT` on the degenerate `order_id`, because an order with several product lines appears on several fact rows. This is the de-duplication approach described in Task 4.

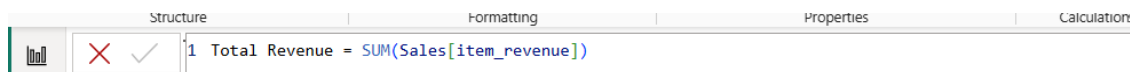


Figure 34: Creating the Total Revenue measure in Power BI

9.4 Report Page 1: Executive Summary

The first page gives a high-level view of overall business performance. It contains:

- Five KPI cards: Total Revenue, Total Orders, Total Units, Average Order Value and Average Units per Order.
- A bar chart of total revenue by customer state.
- A bar chart of total revenue by product category, sorted from highest to lowest.

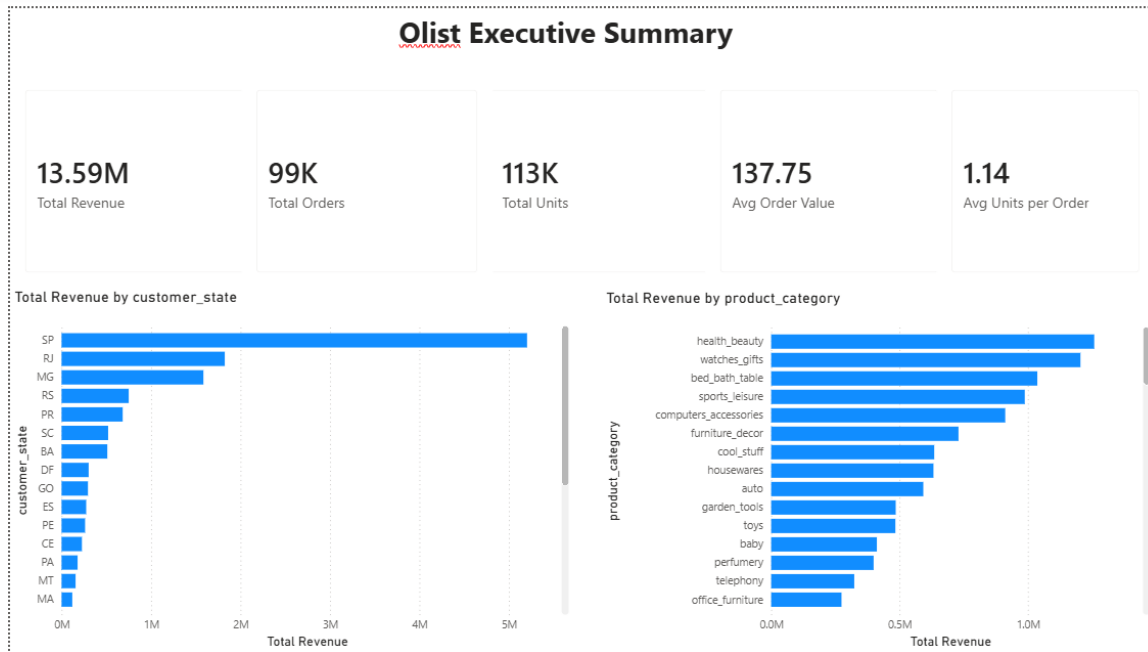


Figure 35: Report Page 1: Executive Summary

9.5 Report Page 2: Trend Analysis

The second page analyses revenue over time and contains:

- A line chart of monthly revenue with one line per year (time-based and comparison analysis).
- A clustered column chart comparing quarterly revenue between 2017 and 2018.

Two data-preparation decisions were needed for this page:

1. **Month ordering.** Month names sort alphabetically by default. `month_name` was set to sort by `month_number` in the `Dates` table, so months appear in calendar order.
2. **Incomplete months.** Inspecting the chart showed near-empty months at the edges of the dataset: September 2016 (267.36), December 2016 (10.90) and September 2018 (145.00), compared with several hundred thousand in a normal month. These are a handful of stray orders, not real trading months, and they made revenue appear to collapse to zero. A page-level date filter (`full_date` on or after 01/01/2017 and on or before 31/08/2018) was applied, and a note on the page states the range. Comparisons therefore use 2017 (twelve months) and 2018 (January to August).

The third quarter of 2018 contains only July and August, so it is a partial quarter and should not be read as a slowdown against 2017.

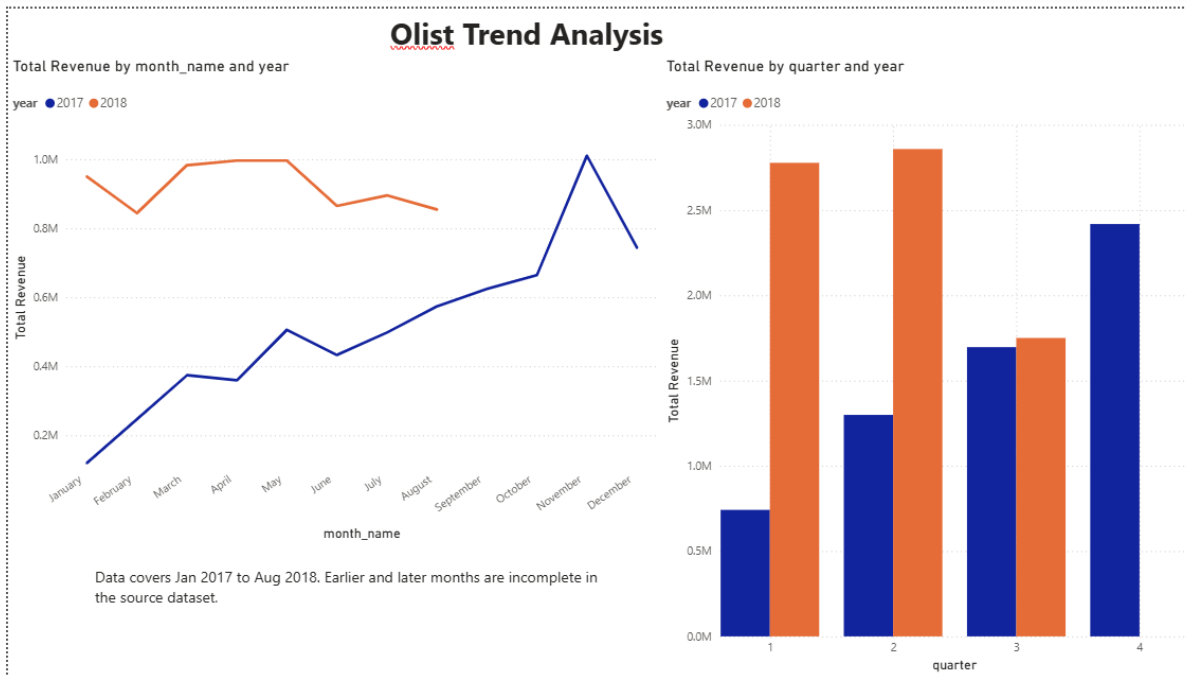


Figure 36: Report Page 2: Trend Analysis

9.6 Report Page 3: Interactive Analysis

The third page supports exploratory OLAP-style analysis and contains:

- Three slicers: year (tile style), customer state (dropdown) and product category (dropdown).
- A column chart of total revenue with a Year → Quarter → Month drill-down hierarchy.
- The same page-level date filter as Page 2.

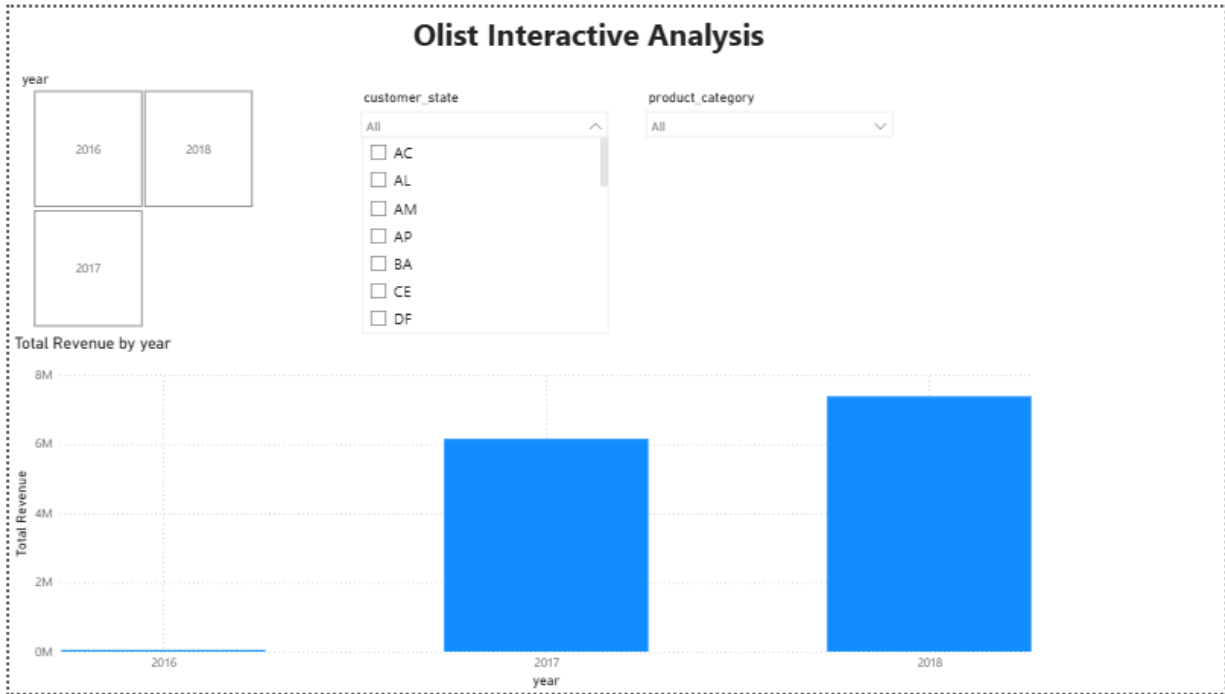


Figure 37: Report Page 3: Interactive Analysis (no filters applied)

9.6.1 OLAP Operations Demonstrated

Table 31 maps each OLAP operation to the feature that demonstrates it.

OLAP operation	How it is demonstrated	Figure
Slice / dice	Year, state and category slicers restrict the chart to a chosen subset of the data	Fig. 38
Drill-down	Clicking a year bar opens it into quarters, then into months	Fig. 39, 40
Roll-up	The drill-up arrow returns the chart from months to quarters to years	Fig. 41

Table 31: OLAP operations and supporting evidence



Figure 38: Slicers in action: customer state AC and category furniture_decor selected

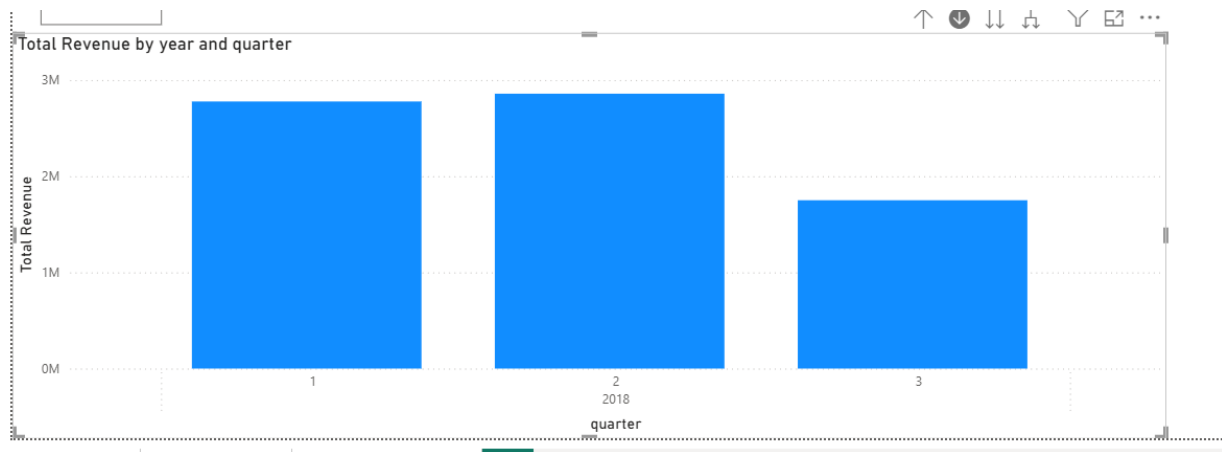


Figure 39: Drill-down from year level to quarter level (2018)

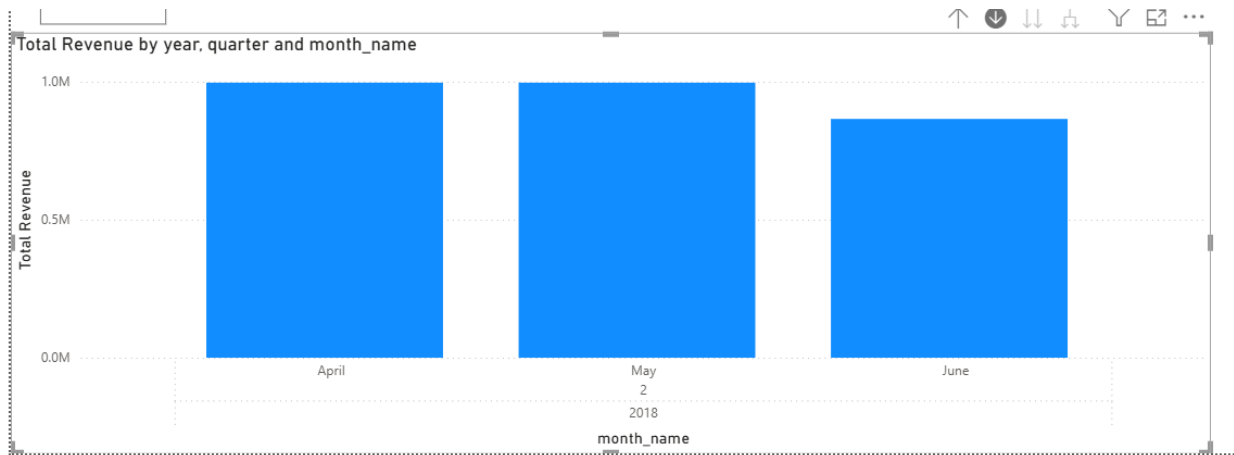


Figure 40: Drill-down from quarter level to month level

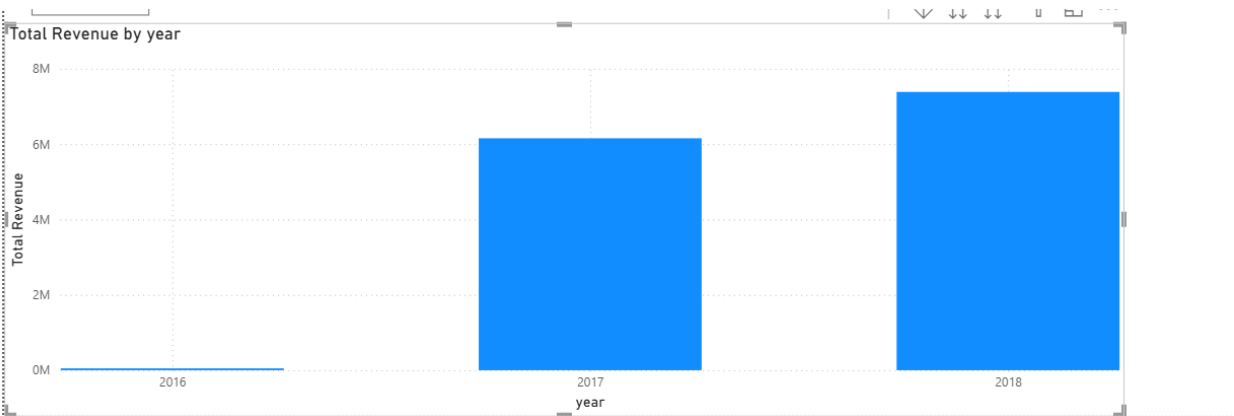


Figure 41: Roll-up back to year level using the drill-up arrow

9.7 Limitations

The `Mart.vw_Sales_Detail` view does not currently expose `review_score`, delivery duration or the late-delivery flag, even though these exist in `Fact_Order_Sales`. The dashboard therefore focuses on sales performance, and customer-satisfaction and delivery KPIs are a natural extension.

10 Task 8: Business Insights and Recommendations

This section presents six insights derived directly from the dashboard in Task 7. Each insight states the evidence, what it means for the business, and a recommendation. All figures are read from the Power BI report. Time-based comparisons use the complete period January 2017 to August 2018 (see Task 7).

10.1 Insight 1: Strong year-over-year growth

Evidence (Page 2). In the first half of 2018, quarterly revenue was about 2.8M in Q1 and 2.85M in Q2, compared with about 0.75M and 1.3M in the same quarters of 2017. That is roughly 3.7 times higher in Q1 and 2.2 times higher in Q2. The eight months of 2018 in the dataset already exceed the revenue of all twelve months of 2017.

Interpretation. The marketplace expanded rapidly during the period, driven by more trading activity rather than a short-term spike.

Recommendation. Plan seller onboarding, warehouse capacity and delivery partnerships for continued growth, because logistics strain is the main risk of fast expansion.

10.2 Insight 2: A clear year-end seasonal peak

Evidence (Page 2). Monthly revenue in 2017 rose through the year and peaked in November at about 1.0M, the highest month of 2017, before falling to about 0.75M in December.

Interpretation. Demand is strongest in the run-up to the year-end shopping season. The dashboard shows the peak but does not prove its cause; a sales event such as Black Friday is a plausible explanation that would need to be confirmed against campaign data.

Recommendation. Prepare stock, seller capacity and promotions ahead of October and November each year, and review December to understand why demand drops so sharply afterwards.

10.3 Insight 3: Revenue is concentrated in one region

Evidence (Page 1). São Paulo (SP) generates about 5.2M of the 13.59M total revenue, roughly 38%. Rio de Janeiro (RJ, about 1.8M) and Minas Gerais (MG, about 1.6M) follow, and every other state is much smaller.

Interpretation. The business depends heavily on one state, which is both a strength and a risk, since local economic or logistics problems in SP would affect a large share of revenue.

Recommendation. Continue to serve SP well, but run targeted marketing and delivery improvements in mid-sized states such as RS and PR to reduce dependence on a single region.

10.4 Insight 4: Revenue is spread across many categories

Evidence (Page 1). The top categories are health_beauty (about 1.25M), watches_gifts (about 1.2M) and bed_bath_table (about 1.05M). Their revenues are close to each other, and no single category dominates.

Interpretation. A varied product mix makes the business less exposed to changes in demand for any one category.

Recommendation. Prioritise inventory and promotion for the top three categories, and use the Page 3 category and state slicers to find categories that sell well in specific states, so promotions can be targeted locally.

10.5 Insight 5: Customers mostly buy a single item

Evidence (Page 1). Orders average only 1.14 units, and the average order value is 137.75.

Interpretation. Most orders contain one item, so revenue growth has come mainly from more orders rather than larger baskets. This suggests room to increase revenue per order.

Recommendation. Introduce product bundles, “frequently bought together” suggestions or free-shipping thresholds slightly above the average order value to encourage customers to add items.

10.6 Insight 6: Monthly growth levelled off in 2018

Evidence (Page 2). Monthly revenue in 2018 stayed between about 0.85M and 1.0M from January to August, roughly the level of the November 2017 peak, and did not show the steady climb seen throughout 2017.

Interpretation. After the rapid growth of 2017, revenue appears to have stabilised at a high level. The dataset ends in August 2018, so it is not possible to say whether this is a longer-term plateau.

Recommendation. Look for new growth levers, such as new regions (Insight 3) and larger basket sizes (Insight 5), rather than relying on the growth pattern seen in 2017.

10.7 How the Insights Support Decision-Making

Together, the insights address three decisions. **Capacity and timing** (Insights 1, 2 and 6) inform when to expand logistics and run promotions. **Where to sell** (Insights 3 and 4) guide regional and category focus, and reduce dependence on São Paulo. **How to grow revenue per customer** (Insight 5) points to basket-building tactics. Because the dashboard is interactive, managers can go beyond these summary views by slicing by state, category and year and drilling from year to month.

10.8 Limitations

The third quarter of 2018 contains only July and August, so Q3 comparisons with 2017 should not be read as a slowdown. The data mart used for the dashboard also excludes review scores and delivery performance, so no insights on customer satisfaction or delivery could be drawn. Adding these fields to the mart view is a recommended extension.

11 Conclusion

11.1 Summary of Work

This project designed and implemented an end-to-end data warehouse and business intelligence solution for the Olist Brazilian e-commerce marketplace. Operational data was extracted into a staging layer in SQL Server and transformed and loaded through an ETL process into a dimensional warehouse. The warehouse is a star schema with one fact table, `Fact_Order_Sales`, and six dimension tables. A data mart view was then built for sales analysis, and an interactive three-page Power BI dashboard was developed on top of it, covering an executive summary, trend analysis and interactive OLAP-style exploration.

11.2 Key Findings

The dashboard showed rapid growth between 2017 and 2018, a clear year-end seasonal peak, a heavy concentration of revenue in São Paulo, a balanced spread across product categories, and a low number of items per order. These findings led to recommendations on capacity planning, regional expansion and increasing basket size.

11.3 Limitations

The dataset covers a limited period, and the months at its start and end are incomplete, so the analysis uses January 2017 to August 2018. Order-level information such as payments is simplified in the fact table by storing the primary payment type. The data mart does not yet include review scores or delivery performance.

11.4 Future Work

The solution could be extended by adding review score and delivery measures to the data mart, adding delivery and estimated-delivery date keys to the fact table, creating a separate payment fact table for payment-method analysis, and scheduling the ETL to run automatically for incremental loads.

Use of AI Tools

In line with the CLEAR framework, AI (Claude, by Anthropic) was used as a supporting tool during this assignment. It was used to discuss dimensional modelling decisions such as fact grain and handling of order-level data, to guide step-by-step use of SQL Server and Power BI, to suggest DAX measure formulas, to help troubleshoot errors during the build, and to help draft and refine the wording of report sections.

All AI suggestions were critically evaluated before use. The SQL scripts, ETL, data mart and dashboards were implemented and run by me. The measure values were checked against the source data, and every figure

quoted in the insights was verified against the dashboard. Screenshots and results in this report come from my own implementation.

12 References

- Olist. Brazilian E-Commerce Public Dataset. Available through Kaggle: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>
- Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3rd ed.). Wiley.